



**Pedro Miguel Afonso
de Pina Monteiro**

**Deteção de Armas em Tempo Real em Imagens de
Videovigilância**

**Real-Time Weapon Detection in Surveillance Video
Footages**



**Pedro Miguel Afonso
de Pina Monteiro**

**Deteção de Armas em Tempo Real em Imagens de
Videovigilância**

**Real-Time Weapon Detection in Surveillance Video
Footages**

*“In the age of technology, surveillance is not about watching but
about predicting human behavior.”*

— Trevor Paglen



Universidade de Aveiro
2024

**Pedro Miguel Afonso
de Pina Monteiro**

**Deteção de Armas em Tempo Real em Imagens de
Videovigilância**

**Real-Time Weapon Detection in Surveillance Video
Footages**

Dissertação apresentada à Universidade de Aveiro para cumprimento dos requisitos necessários à obtenção do grau de Mestre em Engenharia Informática, realizada sob a orientação científica do Doutor Osvaldo Manuel da Rocha Pacheco, Professor Auxiliar do Departamento de Eletrónica, Telecomunicações e Informática da Universidade de Aveiro, e do Doutor Gonçalo José Freitas Carnaz, Professor Adjunto Convidado do Instituto Superior de Contabilidade e Administração da Universidade de Aveiro (ISCA-UA).

o júri / the jury

presidente / president

Prof. Doutor Carlos Manuel Azevedo Costa

professor associado c/ agregação da Universidade de Aveiro

vogais / examiners committee

Prof. Doutora Inês Campos Monteiro Sabino Domingues

professora adjunta do Instituto Superior de Engenharia de Coimbra

Prof. Doutor Osvaldo Manuel da Rocha Pacheco

professor auxiliar da Universidade de Aveiro

**agradecimentos /
acknowledgements**

Agradeço toda a ajuda à minha família e a todos os meus amigos, em especial ao José Trigo por me permitir utilizar a máquina para a realização de testes e treino do modelo. Aos meus orientadores, Professor Doutor Osvaldo Rocha Pacheco e ao Professor Doutor Gonçalo Carnaz pela oportunidade de trabalhar neste projeto e pelo apoio e orientação.

I thank my family and all my friends for all their help, especially José Trigo for allowing me to use the machine for testing and training the model. To my supervisors, Professor Dr. Osvaldo Rocha Pacheco and Professor Dr. Gonçalo Carnaz, for the opportunity to work on project and for their support and guidance.

Palavras Chave

Aplicação da Lei, Armas, Armas de Fogo, Detecção de Armas, Prevenção do Crime, Sistemas de CCTV, Tempo Real

Resumo

Os sistemas de CCTV são cruciais para a vigilância de segurança. No entanto, a sua eficácia é frequentemente comprometida por uma vulnerabilidade: a dependência da monitorização humana, que pode comprometer a observação contínua e eficaz. Trabalhos anteriores neste domínio têm enfrentado vários desafios, especialmente em cenários onde a visibilidade de armas é reduzida e a qualidade da imagem não é a melhor. Este projeto procura ultrapassar esses obstáculos ao melhorar as capacidades de deteção de armas em tempo real nos sistemas de vigilância por CCTV. O objetivo é desenvolver um sistema, denominado SafeGuard, que não só ajude as instituições de segurança a identificar atividades criminosas, mas também permita a análise de dados passados. Ao utilizar tecnologia avançada, este projeto visa tornar os sistemas de vigilância mais eficientes e fiáveis, conduzindo a medidas de segurança mais robustas.

Keywords

CCTV Systems, Crime Prevention, Firearms, Law Enforcement, Real Time, Weapon Detection, Weapons.

Abstract

CCTV systems are crucial for security surveillance. However, their effectiveness is often undermined by a significant vulnerability: the dependence on human monitoring, which can compromise continuous and effective observation. Previous research in this domain has encountered various challenges, particularly in scenarios where weapon visibility is low and image quality is suboptimal. This project seeks to overcome these obstacles by enhancing the capabilities of real-time weapon detection in CCTV surveillance systems. The aim is to develop a system, named as SafeGuard, that not only helps law enforcement agencies in identifying criminal activities but also enables the analysis of historical data. By using advanced technology and new methods, this project aims to make surveillance systems more efficient and reliable, leading to stronger security measures.

**reconhecimento do uso de
ferramentas IA /
recognizing the use of AI
tools**

Reconheço a utilização do Grammarly (Grammarly, <https://app.grammarly.com/>) para correção de erros de gramática e ortografia na língua inglesa, a utilização do YOLOv5 (Ultralytics, <https://github.com/ultralytics/yolov5>) para tarefas de detecção de objetos em vídeos e imagens, a utilização do ChatGPT 3.5 (Open AI, <https://chat.openai.com>) para resumir algumas notas iniciais.

I acknowledge the use of Grammarly (Grammarly, <https://app.grammarly.com/>) for correcting grammar and spelling errors in English, the use of YOLOv5 (Ultralytics, <https://github.com/ultralytics/yolov5>) for object detection tasks in videos and images, and the use of ChatGPT 3.5 (Open AI, <https://chat.openai.com>) to summarize some initial notes.

Contents

Contents	i
List of Figures	v
List of Tables	vii
List of Code Excerpts	ix
Glossary	xi
1 Introduction	1
1.1 Context and Motivation	1
1.2 Objectives	3
1.3 Outline of the Document	4
2 Concepts and Related Work	5
2.1 Domain-related Concepts and Technologies	5
2.1.1 Weapons	5
2.1.2 Closed Circuit Television (CCTV)	6
2.1.3 CCTV Setup and Configuration	7
2.2 Object Detection	8
2.2.1 General Concepts	8
2.2.2 Evolution of Object Detection Methods	9
2.2.3 Performance Metrics	10
2.2.4 Algorithms and Techniques Reviews	11
2.3 Related Work	12
2.3.1 Research Method	12
2.3.2 Analysis of Datasets in Weapon Detection	13
2.3.3 Architectural Approaches in Weapon Detection	14
2.3.4 Comparative Analysis of Weapon Detection Technologies	16

3	Architecture, System Design and Requirements	21
3.1	Basic Use Cases	21
3.2	Functional Requirements	22
3.3	High Level Architecture	22
3.4	Non-Functional Requirements	24
4	You Only Look Once (YOLO) Model for Video Analysis	25
4.1	YOLO Versions Overview	25
4.2	YOLOv5 Model Overview	26
4.3	Dataset Proposed	28
5	SafeGuard System	31
5.1	Architecture Overview	31
5.2	Folder Structure	32
5.3	Video Retrieval and Management	33
5.3.1	RabbitMQ Setup	33
5.3.2	Message Queuing and Processing	34
5.4	Frontend Development	34
5.4.1	Views Implementation	34
5.4.2	Integration with Docker	36
5.4.3	Testing and Quality Assurance	37
5.5	Backend Development	38
5.5.1	Authentication and Security	38
5.5.2	Client-Server Interaction Flows	39
5.5.3	Data Models	39
5.5.4	API Endpoints	40
5.5.5	Django Consumers	42
5.5.6	Database Integration	43
5.5.7	YOLO Model Integration	43
5.6	Web Server Configuration	43
5.6.1	Nginx Setup and Load Balancing	43
5.7	Monitoring and Visualization	44
5.7.1	Metrics Collection with Prometheus	44
5.7.2	Visualization using Grafana	45
5.8	Implementation Challenges	45
5.9	Scalability and Performance Optimizations	46
6	Tests and Results	47
6.1	YOLO Model Results Evaluation	47

6.2	Real-World Scenario Testing	50
6.2.1	Non-Weaponized Scenario Analysis	50
6.2.2	Weaponized Environment Detection	51
6.3	User Experience Evaluation	51
6.4	Comparative Analysis	52
7	Conclusions and Future Work	53
7.1	Conclusions	53
7.2	Work Contributes	54
7.3	Future Work	54
	References	55

List of Figures

1.1	Active Shooter Incidents Casualties in the United States (US) [7]	1
1.2	Active Shooter Incidents in the US [7]	1
1.3	Firearm Used by Active Shooters in the US [7]	2
1.4	Crime Rates in Portugal from 2014 to 2022 [11]	2
1.5	Geographical Distribution of Crime Rate Increase in Portugal by District in 2022 [11] . .	2
2.1	Melee Weapon Example	6
2.2	Different Firearms Example	6
2.3	CCTV Cameras (Left Image) and CCTV Footage Samples (Right Image)	6
2.4	Semi-covert Dome Camera	7
2.5	Covert Cameras	7
2.6	Labeled Graph H (Simple Graph Representation of District 5) [18]	8
2.7	Object Detection Image [21]	9
2.8	Object Detection Roadmap [22]	9
2.9	Intersection over Union [25]	11
2.10	Shenoy <i>et al.</i> [1] System Architecture	15
2.11	Gawade <i>et al.</i> [2] Architecure Diagram	15
2.12	Al-Mousa <i>et al.</i> [3] System Diagram	16
2.13	Object Recognition to Detection Hierarchy [5]	16
2.14	Al-Mousa <i>et al.</i> [3] Image Pre-processing Flowchart	18
2.15	Hnoohom <i>et al.</i> [4] Weapon Detection Process	18
2.16	Distribution of Models Used in Studies	19
2.17	Distribution of Metrics Used in Studies	19
3.1	System Basic Use Cases [38]	21
3.2	High Level Architecture Proposal	23
4.1	YOLO Versions Timeline [40]	25
4.2	YOLOv5 Architecture [42]	26
4.3	CSPNet - Cross Stage Partial Network	27

4.4	Feature Pyramid Network	27
4.5	Dataset Creation Steps	28
5.1	Implementation Architecture Overview	31
5.2	Folder Structure Diagram	33
5.3	RabbitMQ Processing [52]	34
5.4	Single and Multi Camera Pages	35
5.5	Detections History Page	35
5.6	Upload Video Feature	36
5.7	Authentication Flowchart	38
5.8	Basic Client-Server Interaction Flows	39
5.9	Database Schema	40
5.10	User Views	41
5.11	Detections Views	41
5.12	Uploaded Video Views	41
5.13	Grafana Dashboard	45
6.1	Recall Variation with Epochs	48
6.2	Precision-Recall Curve	49
6.3	F1-Confidence Curve	49
6.4	Confusion Matrix	50
6.5	Video 1 Extracted Frame [44]	51
6.6	Video 2 Extracted Frame [44]	51
6.7	Annotated Frame from Weaponized Environment [44]	51
6.8	Frame From Weaponized Scenario [44]	51
6.9	Comparative Analysis with Related Works	52

List of Tables

2.1	Dataset images classes number [31]	13
2.2	Datasets created by Bhatti <i>et al.</i> [5]	13
2.3	Models Performance Comparison [33]	17
2.4	Moran <i>et al.</i> [6] Model General Results	17
2.5	Authors models and results	19
6.1	Training Parameters and Performance Metrics	48
6.2	Optimal Configuration Parameters and Results	49

List of Code Excerpts

1	Login Test.	37
2	Cuda Initialization.	42
3	Nginx Configuration Setup	44

Glossary

CCTV	Closed Circuit Television	Faster-RCNN	Faster Region-based Convolutional Neural Networks
AI	Artificial Intelligence	DCNNs	Deep Convolutional Neural Network
IoU	Intersection over Union	CNN	Convolutional Neural Network
BB	Bounding Box	DL	Deep Learning
US	United States	GPU	Graphical Processing Unit
AP	Average Precision	TPU	Tensor Processing Unit
mAP	Mean Average Precision	CUDA	Compute Unified Device Architecture
YOLO	You Only Look Once	E2E	End-to-End
PANet	Path Aggregation Network	DOM	Document Object Model
FPN	Feature Pyramid Network		
SSD	Single Shot Detection		

Introduction

This chapter starts by explaining the context and motivation behind the work. It outlines the primary objectives, offering a comprehensive understanding of the work's aims. Additionally, provides a detailed overview of the document's structure.

1.1 CONTEXT AND MOTIVATION

In recent years, there has been an alarming increase in firearm-related crime. In 2022, the Federal Bureau of Investigation reported fifty active shooter incidents in the US. These were spread across 25 states, and 313 casualties in these incidents, Figure 1.1, including 100 deaths and 213 injuries, with the most severe incident causing seven fatalities and 48 injuries [7].

Notably, there was a 18% decrease in such incidents in 2021, but a significant 66.7% increase compared to 2018, as shown in Figure 1.2.

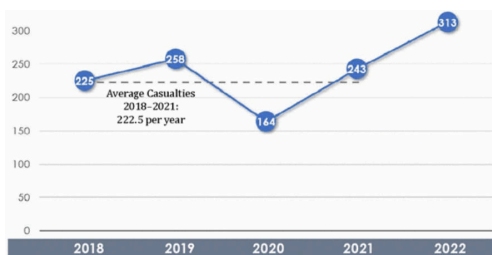


Figure 1.1: Active Shooter Incidents Casualties in the US [7]

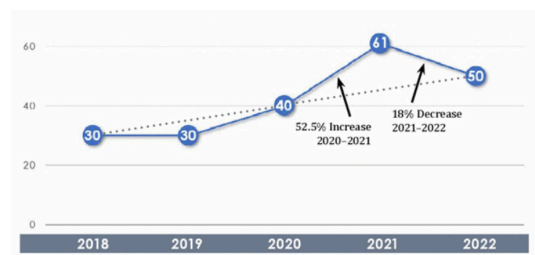


Figure 1.2: Active Shooter Incidents in the US [7]

Despite the existence of strict gun laws in many countries, firearm access remains relatively easy [8], [9]. The US is the global leader in civilian gun ownership, with an estimated 120.5 firearms per 100 people, which is nearly twice as high as the country with the second-highest rate. [10].

Within the 50 active shooter incidents in 2022, a total of 61 firearms were utilized, as shown in Figure 1.3. Handguns were the most frequently used, accounting for 29 of the weapons, followed closely by rifles at 26.

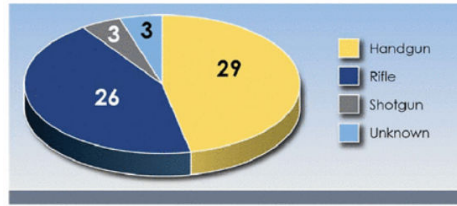


Figure 1.3: Firearm Used by Active Shooters in the US [7]

In relation to Portugal, as RASI [11], reports, there has been a substantial increase in criminal activity. The total number of general crime reports registered by the Public Security Forces was 343845 in 2022, which is an increase of 42451 compared to 2021, representing a 14.1% rise. Specifically, the total number of reported violent and serious crimes was 13281, up by 1667 from the previous year, marking a 14.4% increase, as shown in Figure 1.4.

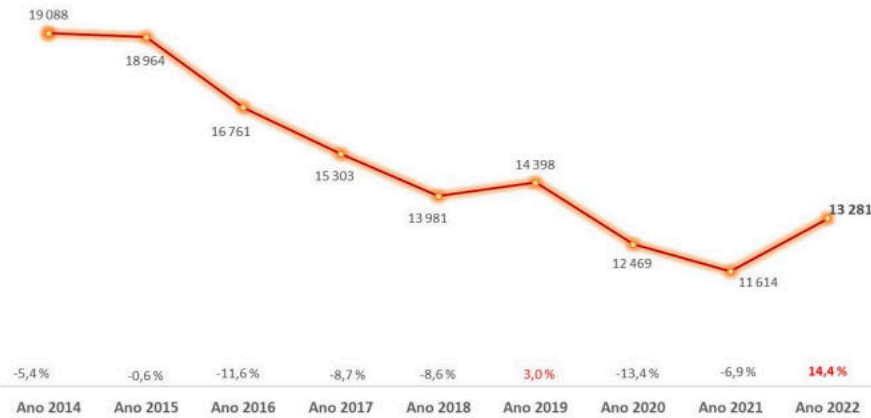


Figure 1.4: Crime Rates in Portugal from 2014 to 2022 [11]

Figure 1.5 shows a geographically decrease in the districts of Braga (-5.1%), Bragança (-17.9%), and Vila Real (-10%). Conversely, there was a significant increase in Lisbon (+11.4%), Porto (+11.3%), Faro (+23.9%), and the Autonomous Region of Madeira (+52%).

Participações registadas por distrito e região autónoma									
Distritos	Ano 2021	Ano 2022	Var %	Dif	Distritos	Ano 2021	Ano 2022	Var %	Dif
✗ Aveiro	400	470	17,5 %	70	🟡 Portalegre	93	93	0,0 %	0
✗ Beja	84	120	42,9 %	36	✗ Porto	1 886	2 100	11,3 %	214
🟢 Braga	469	445	-5,1 %	-24	✗ Santarém	314	375	19,4 %	61
🟢 Bragança	56	46	-17,9 %	-10	✗ Setúbal	1 234	1 330	7,8 %	96
✗ Castelo Branco	106	148	39,6 %	42	✗ Viana do Castelo	102	124	21,6 %	22
✗ Coimbra	252	317	25,8 %	65	🟢 Vila Real	100	90	-10,0 %	-10
✗ Évora	93	136	46,2 %	43	✗ Viseu	153	158	3,3 %	5
✗ Faro	707	876	23,9 %	169	✗ Madeira	177	269	52,0 %	92
✗ Guarda	56	60	7,1 %	4	✗ Açores	220	232	5,5 %	12
✗ Leiria	286	344	20,3 %	58	✗ Não especificado	207	402	94,2 %	195
✗ Lisboa	4 619	5 146	11,4 %	527	✗ Total	11 614	13 281	14,4 %	1 667

Figure 1.5: Geographical Distribution of Crime Rate Increase in Portugal by District in 2022 [11]

Along the years, it was noticed that CCTV systems have become a crucial tool for crime prevention. Research indicates a correlation between CCTV deployment and a reduction in crime rates, especially in car parks [12]. For example, in Florida, the implementation of this

systems has resulted in crime rate reductions of up to 51% in certain areas [13]. Alongside its growing popularity worldwide, CCTV surveillance has generated debates concerning its effectiveness, efficiency, and related privacy issues. However the CCTV surveillance frequently raise concerns about the invasion of privacy and the risk of violating privacy laws [14].

A significant challenge in operating CCTV systems is the reliance on human monitoring, which is susceptible to errors due to fatigue or distraction. Additionally, the financial implications of CCTV, including the costs associated with constructing and maintaining these surveillance systems, form an essential part of the discussion. These costs must be weighed against the potential benefits in crime reduction and public safety [14].

The effectiveness of CCTV in controlling crime varies and is influenced by several factors, including geographic location, crime types, and the strategies employed for camera monitoring. Studies have revealed that CCTV operations managed by private security personnel tend to have a more substantial crime prevention impact compared to those overseen by police [15]. In Sweden’s deprived neighborhoods, the use of CCTV has led to a decrease in violent crimes, though it has not significantly affected property crime rates or the clearance of crimes [16].

CCTV systems have emerged as key tools in crime prevention, offering the ability to reduce specific types of crime, especially in environments where they are actively monitored and used alongside other interventions [16].

1.2 OBJECTIVES

The primary goal of this work is to advance in the field of public safety and security through the development of a Deep Learning-Based system for real-time weapon detection in CCTV surveillance footage, designed for the detection and classification of weapons, such as firearms and knives. This includes several key aspects:

- Exploration of Current Technologies and Methods: thorough review of the state-of-the-art technologies and methodologies used in real-time, analyzing existing approaches’ effectiveness, efficiency, and applicability in various real-world scenarios;
- Identification and Analysis of Public Databases: identify public databases that contain relevant data on anomalous objects;
- Development of a Deep Learning Model: develop a state-of-the-art deep learning model capable of accurately identifying and classifying firearms or knives in diverse and challenging surveillance scenarios;
- Architecture Design and Prototyping: proposing and developing a prototype solution, named as SafeGuard, with a well-defined architecture;
- Performance Evaluation and Limitation Analysis: extensive testing of the prototype in various real-life scenarios to evaluate its performance, accuracy, and reliability;

The expectation is that the application and model developed through this research will surpass existing solutions in terms of efficiency, accuracy, and practicality. To achieve this, it is imperative to conduct a comprehensive analysis of the work done by other researchers. This examination will not only provide insights into the most effective strategies and methodologies but will also shed light on the common challenges and barriers encountered in similar attempts.

1.3 OUTLINE OF THE DOCUMENT

The document is organized into subsequent chapters as follows:

- Chapter 2 offers a detailed analysis of the existing research and technologies in weapon detection, focusing on critical aspects like weapon categorization, the significance of CCTV, and the application of deep learning algorithms;
- Chapter 3 discusses the system requirements, along with various use cases that the project addresses. It also outlines the high-level architecture of the system, describing the overall structure and key components;
- Chapter 4 offers a comprehensive review of the dataset created for the project, detailing the data collection methods, preprocessing techniques, and the characteristics of the dataset. It also covers the object detection model used, discussing its architecture and training process;
- Chapter 5 focuses on the implementation of the safeguard system. It provides an in-depth examination of each module within the system, addressing the main challenges encountered during development and the performance optimizations applied;
- Chapter 6 presents the results of the implemented system. It evaluates the model's performance and accuracy and offers a detailed analysis of the user experience;
- Chapter 7 concludes the thesis by summarizing the key findings of the research, and suggests directions for future work.

Concepts and Related Work

This chapter presents the concepts and technologies related to the application domain and the related work that could be used to inform our options for an approach. The discussion begins with an overview of the general context in Section 2.1, laying the groundwork for a more detailed examination of weapon detection technologies and CCTV systems. Section 2.2 delves into object detection, reviewing key algorithms and techniques. Section 2.3 focuses on related research, including methodologies and weapon detection analysis, and a comprehensive examination of datasets and architectural methods in weapon detection.

2.1 DOMAIN-RELATED CONCEPTS AND TECHNOLOGIES

These sections support the background knowledge needed to understand some of the concepts and technologies in use along our work.

2.1.1 Weapons

In order to have a system that detects weapons on video footage, we need an understanding of a basic concept: weapons types and definition. Therefore, the Cambridge Dictionary¹ defines a weapon as "any object used in fighting or war, such as a gun, bomb, knife, etc.". This definition implies that a weapon is a tool or instrument that is intended to cause damage or injury, whether to living beings, structures or systems. Weapons have a wide range of uses, such as hunting, self-defense or warfare. However, their fundamental purpose remains unchanged: they amplify the user's ability to exert force, in either an offensive or defensive capacity.

They are categorized based on different criteria: range, mechanism and intended use. However, two categories, firearms and close-combat weapons, will be the main focus of this study.

¹<https://dictionary.cambridge.org/dictionary/english/weapon> [Accessed on: November,2023]

Melee weapons are close-combat instruments that require the user to be in direct proximity to their target, like swords, daggers, maces, and clubs.

Firearms are a subset of ranged weapons that discharge projectiles powered by rapidly expanding high-pressure gas from chemical reactions. They can be categorized into:

- Handguns: Small, handheld firearms like pistols and revolvers.
- Rifles: Designed for accuracy, rifles have a longer barrel and are often used in situations requiring precision.
- Shotguns: These fire shells that contain multiple pellets, making them effective at close range.
- Automatic and Semi-Automatic: Automatic firearms continuously fire bullets as long as the trigger is pressed, while semi-automatics require a trigger pull for each shot.

Figure 2.1 shows an example of a melee weapon, while Figure 2.2 presents various types of firearms, showcasing the diversity in firearm design and functionality.



Figure 2.1: Melee Weapon Example



Figure 2.2: Different Firearms Example

2.1.2 CCTV

CCTV refers to a video surveillance system with strategically positioned cameras in various locations. These cameras capture footage, exemplified in Figure 2.3, which is then transmitted to monitors for both real-time observation and later playback.



Figure 2.3: CCTV Cameras (Left Image) and CCTV Footage Samples (Right Image)

The primary goal of a CCTV system is to enhance area security through persistent surveillance of key areas. This is highly beneficial for large spaces or sites containing valuable

or sensitive items. Besides recording, these systems can issue alerts for detected motion, especially during off-hours in vacant business premises. These alerts are crucial for quickly informing authorities about possible security threats.

Additionally, surveillance systems are instrumental in monitoring activities during both operational and non-operational hours and serve as crucial tools in identifying suspects in criminal activities.

2.1.3 CCTV Setup and Configuration

Ensuring the effectiveness of a CCTV security system requires careful consideration of several key factors:

- Environmental Context Analysis:
 - Understand the specific crime dynamics of the area;
 - Strategically position cameras to cover high-risk zones;
- Line-of-Sight Optimization:
 - Ensure clear visibility for each camera;
 - Minimize obstructions such as immovable objects and foliage;
- Regular Maintenance:
 - Clear any new obstructions that may arise;
 - Perform routine checks and updates to maintain system functionality.

The design of the cameras significantly influences their ability to prevent crime. Semi-covert 'dome' cameras, like the camera showed in Figure 2.4, are often more effective in discouraging criminal activities compared to overt cameras, Figure 2.5, making them an ideal option in many scenarios. An effective CCTV system is not just about installation but also involves active monitoring and prompt enforcement actions [5].



Figure 2.4: Semi-covert Dome Camera



Figure 2.5: Covert Cameras

Keval and Sasse [17] identified a spectrum of problems in control room environments, specifically in the management and operation of CCTV systems. A critical aspect is the poor technology configuration in these environments, which includes improperly positioned cameras in low-crime zones or areas with blind spots, and the adverse effects of weather on analog signals. Additionally, outdated or malfunctioning equipment, like broken camera controllers, further impairs surveillance effectiveness. Also, the quality of video recordings was another area of significant concern. Many control rooms recorded footage at low resolutions, rendering it ineffective for detailed analyses or criminal investigations.

Gaylon *et al.* [18] proposed a model which delineates a community's surveillance blueprint, marking the prime locations for CCTV installation.

- Red dots represent primary entry and exit points, ensuring close monitoring of individual movements through these areas;
- Black dots are placed at street intersections and corners, providing broad surveillance coverage.

In the model described in Figure 2.6, the cameras positioned at red dots would be pivotal for controlling access points, while those at black dots would enable panoramic monitoring of the area, essential for observing multiple streets and deterring potential criminal activity.

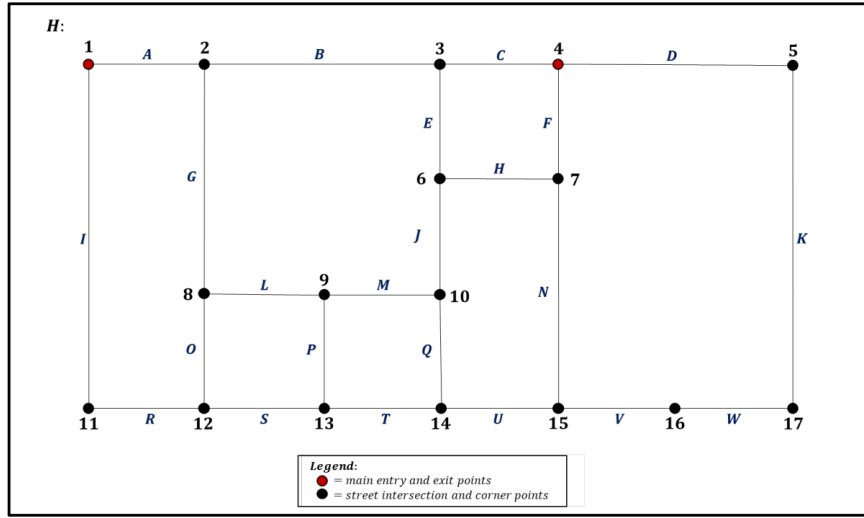


Figure 2.6: Labeled Graph H (Simple Graph Representation of District 5) [18]

2.2 OBJECT DETECTION

This section covers the basics of object detection, explaining its definition, exploring different methodologies used in the field, and outlining key metrics for evaluating performance.

2.2.1 General Concepts

Zhao *et al.* [19] describe object detection goal as "to determine where objects are located in a given image (object localization) and which category each object belongs to (object classification)", undertaking the dual challenge of not only locating an object within the visual frame, creating a bounding box around it, but also determining its category. Kaur and Singh [20] further reinforce this stating "Object detection is a fusion of object location and object classification task".

Object detection is the process of identifying objects in images and videos. The process of detecting these objects is fundamentally rooted in the training of object detection models. There are several key steps in this crucial training phase:

- **Data Collection:** the purpose of this task is to build a set of images, each showcasing the object in a variety of contexts, including different backgrounds and angles;

- **Labeling:** this task aims to annotate (manually or automatically) each image with labels that reflects the object classes, such as the presence of a certain type of object;
- **Feature Learning:** enable a trained model to recognize unique object features such as shape, color, texture, and patterns, these characteristics must be learned. This task is performed using labeled images (the outcome of the previous task);
- **Generalization:** the goal of this task is the generalization of object detection, not only for labelled images, but also for new (unseen) images, exemplified in Figure 2.7. A set of metrics make up this task to allow for accuracy when training on unseen objects (images or not).

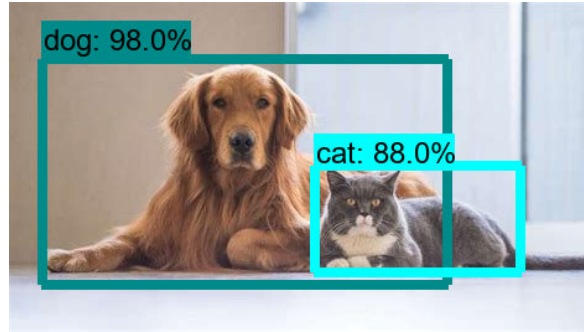


Figure 2.7: Object Detection Image [21]

2.2.2 Evolution of Object Detection Methods

The 2014 milestone, Figure 2.8, marks a before-and-after in Artificial Intelligence (AI) applications [22]. Before, the field predominantly utilized traditional methods for tasks like object detection. These methods involved a multi-step process beginning with the selection of regions in images, which was often inefficient and computationally expensive due to the use of techniques like multi-scale sliding windows. Following this, feature extraction methods such as HOG, Haar-like, and SIFT were employed, but these struggled with variability in backgrounds and lighting conditions. The final step, classification, relied on algorithms like Adaboost, but these traditional techniques frequently fell short in terms of accuracy and efficiency [20].

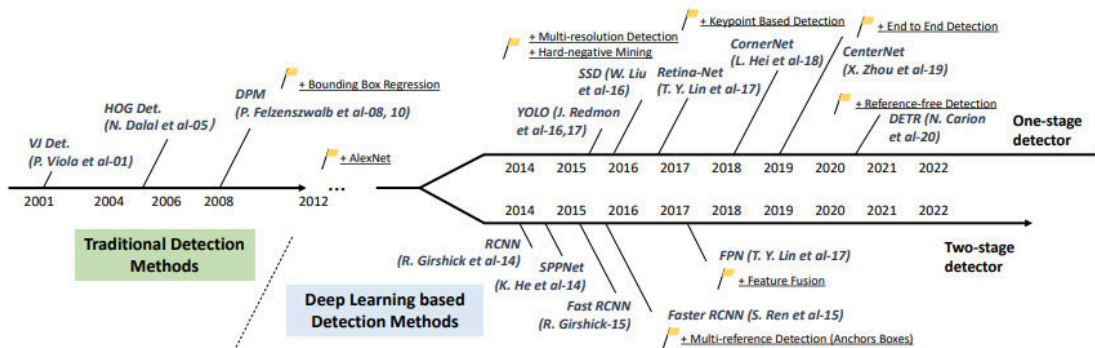


Figure 2.8: Object Detection Roadmap [22]

After 2014, the landscape of AI research underwent a significant transformation with the advent of deep learning-based methods. These new techniques, including Faster Region-

based Convolutional Neural Networks (Faster-RCNN), Single Shot Detection (SSD), and YOLO, revolutionized object detection by automatically learning feature representations from data. This shift not only addressed the limitations of the traditional methods, such as high computational costs and inadequate feature representation, but also led to substantial improvements in both the accuracy and efficiency of object detection processes.

As seen in Figure 2.8 there are two primary methodologies used for identifying and classifying objects. These methodologies differ significantly in their operational approach and performance characteristics.

There is the **two-stage** detection process that operates in a sequential two-phase process. The process involves two key steps: "(i) use a Region Proposal Network to generate regions of interests in the first stage and (ii) send the region proposals down the pipeline for object classification and bounding-box regression"[23]. The two-stage framework ensures high accuracy but at the cost of speed.

In contrast, **one-stage** detectors treat object detection as a simple regression problem, directly analyzing the image to determine class probabilities and bounding box coordinates. This approach is faster but generally achieves lower accuracy rates compared to two-stage detectors [24].

2.2.3 Performance Metrics

In the field of object detection a variety of metrics are employed to evaluate the performance of detection models. These metrics play a crucial role in both training and testing stages, optimizing classifiers, and measuring their efficiency.

Precision is calculated as the percentage of correctly identified objects (true positives) among all detections (true and false positives). High precision indicates a low rate of false positives. Recall assesses the model's ability to detect all relevant objects in the dataset. It is the percentage of correctly detected objects (true positives) among all actual objects (sum of true positives and false negatives). High recall implies that the model is effective in identifying most of the relevant objects [25].

Following this, accuracy measures the overall correctness of the model, reflecting the proportion of all true results (true positives and true negatives) against the total number of cases.

The Average Precision (AP) metric integrates precision and recall at different confidence thresholds. It addresses the trade-off between precision and recall, considering detections above a confidence level. AP is calculated based on the area under the precision-recall curve [25]. Mean Average Precision (mAP) averages the AP across all classes within a database, crucial for assessing multi-class detector performance [26].

F1 score is used to estimate the balance between recall and precision. It is particularly valuable in situations where there is an imbalanced class distribution, as it provides a single metric that combines both precision and recall. Is defined as the harmonic mean² of precision

²type of average that gives equal weight to both precision and recall, unlike the arithmetic mean which can be disproportionately influenced by high values of one metric over the other

and recall, as represented in Expression 2.1.

$$F1 \text{ score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.1)$$

A high F1 score indicates both high precision and high recall. This metric is particularly useful in the context of binary classifiers on unevenly distributed datasets, where it provides a more balanced view of a model's performance compared to using precision or recall alone [20].

Intersection over Union (IoU) measures the overlap between the predicted Bounding Box (BB) and the ground truth BB, Figure 2.9. It's calculated as the ratio of the intersection area to the union area of these BBs. The resulting IoU value ranges between 0 and 1, with values closer to 1 indicating more accurate detection. "The IoU, based on the Jaccard Index, measures the overlap between the predicted bounding box and the ground-truth bounding box." [26].

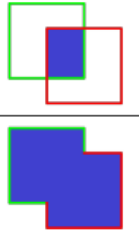
$$\text{IOU} = \frac{\text{area of overlap}}{\text{area of union}} = \frac{\text{area of overlap}}{\text{area of union}}$$


Figure 2.9: Intersection over Union [25]

2.2.4 Algorithms and Techniques Reviews

This section analyzes current object detection algorithms and methodologies, drawing on a range of studies and innovations from various online repositories and databases. It focuses on selected reviews³ that detail their development, methodologies, outcomes, and contributions to the advancement of object detection. This analysis provides insights into the evolution of object detection techniques from their inception to modern applications.

Zhao *et al.* [19] provide a review of deep learning-based object detection methods, beginning with the historical context of deep learning and the rise of Convolutional Neural Network (CNN). The review covers key models such as R-CNN, which uses selective search for region proposals and CNNs for feature extraction, and its successors SPP-net, Fast R-CNN, and Faster-RCNN, which enhance efficiency and accuracy through integrated region proposal networks and streamlined processing pipelines. YOLO introduces a real-time detection approach by treating detection as a single regression problem, while SSD improves small object detection by using multiple feature maps at different scales. The authors emphasize that end-to-end training frameworks like Faster-RCNN significantly boost performance and efficiency, while multi-scale approaches like SSD effectively address the challenge of detecting smaller objects.

Srivastava *et al.* [27] provide a comprehensive review of image processing techniques, emphasizing machine learning applications in visual interpretation. The study begins by

³comprehensive overview or evaluation of existing literature on a particular topic

highlighting the foundational aspects of computational vision, particularly the significance of array-based media computation in the digital era. It then examines key image processing algorithms, including SSD, Faster-RCNN, and YOLO, detailing their architecture, methodologies, and distinguishing features. The researchers conduct systematic experiments using datasets like Microsoft COCO [28], comparing these algorithms to elucidate their respective strengths and limitations.

Kaur and Singh [20] emphasize the intricacies of computer vision in their review, highlighting the significant impact of Deep Convolutional Neural Network (DCNNs). Their study spans various applications, from video processing to speech recognition, with a particular focus on object detection's importance in areas like transportation and security. They detail crucial evaluation metrics, especially AP, for assessing detection effectiveness. Notably, they mark the pivotal shift to deep learning techniques post-2014, analyzing frameworks like SSD and YOLO. By comparing these newer methods with traditional ones and evaluating their performance on datasets like PASCAL VOC [29] and MS COCO [28], they offer insights into the dynamic field of object detection.

Alzubaidi *et al.* [30] review Deep Learning (DL), highlighting its rapid growth and superior performance in various fields such as cybersecurity, NLP, bioinformatics, robotics, and medical information processing. The review focuses on CNNs, detailing their evolution from AlexNet to High-Resolution Networks and their advantages, such as automatic feature extraction. The authors address challenges in DL, including the need for large datasets, model interpretability, and overfitting, proposing solutions like data augmentation and advanced optimization algorithms. They also discuss the impact of computational tools like Graphical Processing Unit (GPU) and Tensor Processing Unit (TPU) on DL performance. The review further highlights the diverse applications of DL, showcasing its often superior, sometimes human-competitive, performance.

2.3 RELATED WORK

In this section, we systematically review various object detection architectures that could be adapted to our approach.

Therefore, a comprehensive overview of these contributions is presented, organized into focused areas: research methodologies, an in-depth comparative analysis of weapon detection technologies, a thorough exploration of datasets pertinent to this field, and a review of the various architectural approaches employed in weapon detection systems.

2.3.1 Research Method

This section presents a comprehensive review of critical research in the field of weapon detection. The review is structured into three key areas: insights from other researchers' datasets and system architectures. Due to the wide range and variety of articles from different fields, a careful selection method was used. The search, conducted on Scopus, ResearchGate, and arXiv, focused on keywords like "deep," "learning," "real," "time," "weapon," and "detection." This search initially produced a large number of articles. A detailed and strict selection

process was then applied to choose the most relevant articles that significantly relate to the main theme, resulting in a final count of 16 documents.

2.3.2 Analysis of Datasets in Weapon Detection

The datasets are a crucial element for this kind of tasks, because they mimic real-world situations. Therefore, the gathering of datasets is a relevant baseline to achieve valid results in a object detection task. In Rehman and Fahad [31] study, the dataset comprises three distinct classes: short guns, long guns, and knives - Table 2.1. This dataset is gathered from a variety of platforms, including surveillance videos from YouTube, simulations of firearms and knives created within the Unity framework, and images of pistols, revolvers, and knives from the Open Images Dataset as well as Kaggle. For video, a set of individual frames was extracted and manually labeled using the LabelImg Python utility. To enhance the model's training performance, data augmentation techniques were employed.

Table 2.1: Dataset images classes number [31]

Short guns	Long guns	Knives	Total
4370	3426	1984	10567

Bhatti *et al.* [5] developed datasets for real-time weapon detection, focusing on pistols in various settings. Data was sourced from the internet, YouTube, GitHub, research groups, and the Internet Movie Firearm Database [32], leading to three datasets categorized into 'Pistol' and 'Not-Pistol' classes, Table 2.2. The 'Pistol' class contained handheld weapons like pistols and revolvers, while 'Not-Pistol' included objects often confused with weapons, like wallets and cell phones. The researchers standardized image size and resolution during data preparation, applied mean normalization, and used data augmentation to enhance the dataset, aiding the model's generalization capabilities.

Table 2.2: Datasets created by Bhatti *et al.* [5]

Sr.No.	Category	Total Data	Training Data	Test Data	Split Size
1.	Dataset 1	1732	1251	260	15%
2.	Dataset 2	5254	3797	784	15%
3.	Dataset 3	8327	7328	999	12%

Thangaraj *et al.* [33] used Google Collab [34] to train models on 1496 images with pre-trained COCO dataset weights [28]. The images were divided into 1200 for training and 296 for validation and testing (148 each). Training lasted 40 epochs, taking about 120 minutes, with images sized at 416x416 pixels. Both models, within the PyTorch framework, utilized a COCO pre-trained backbone feature extraction network, which was fine-tuned during training to better represent handguns.

Gawade *et al.* [2] used a comprehensive dataset consisting of 10014 distinct weapon images to train and evaluate the model's accuracy. These images were categorized into three primary object types: knives, with a total of 3641 images; long guns, comprising 2497 images; and small guns, which included 3876 images. The dataset was strategically divided, allocating

80% of the images (8011 images) for training and the remaining 20% (2003 images) for testing. Each image used had a resolution of 240 x 240 pixels, and during the training phase, they were processed in batches of 32.

Shenoy *et al.* [1] and Moran *et al.* [6] both utilized the DaSCI Weapon Dataset [35] for weapon detection research, with Shenoy *et al.* [1] focusing on differentiating weapons from ordinary objects and training their system on 91 weapons. Additionally, employed the CAVIAR Dataset [36] for Suspicious Behavior Prediction, using its video clips of six human actions, annotated in XML, to classify actions as suspicious or not. They parsed XML files to align annotations with video frames for analysis. Moran *et al.* [6] used the DaSCI Dataset [35] for knife detection, comprising 2,078 images, primarily from the internet and YouTube. They also utilized the MS COCO 2017 Dataset [28], a larger dataset with 330,000 images in 80 classes, including 4,326 images labeled under the 'knife' class.

Safa and M [37] faced challenges in gathering a comprehensive weapon dataset, particularly for knife and screwdriver images, due to the lack of labeled datasets suitable for classifier training. To overcome this, they used web scraping to gather images from websites and GitHub. These images were manually labeled with tools like LabelImg and Roboflow, and standardized to a 416 x 416 resolution. Their final dataset contained about 6,000 images, with 2,000 each of pistols, knives, and screwdrivers. For training and testing their models, they divided the images in an 80:20 ratio.

Al-Mousa *et al.* [3] installed a Raspberry Pi B+ with a camera 1.8m high to capture images of consenting students with handgun replicas at a lab entrance. They initially collected 14k images at 1920x1080 resolution, which were expanded to 28k after data augmentation. A challenge in their study was the small size of the guns in the images, posing a risk of misclassification.

Hnoohom *et al.* [4] used the ARMAS Weapon Detection Dataset and IMFDB Weapon Detection System [32] for firearm-related crime image collection. The datasets included 3000 and 4940 images of pistols and rifles, respectively. These images underwent preprocessing to fit object detection standards and were split into training and testing sets in an 80:20 ratio.

2.3.3 Architectural Approaches in Weapon Detection

The evolution of real-time weapon detection systems has been significantly influenced by diverse architectural approaches proposed by various researchers. This section delves into a brief analysis of some of these architectures.

Safa and M [37] architecture process begins collecting a large set of images that will be used to teach the detection model what various weapons look like. Each image is then labeled to identify where the weapons are. Once the images are labeled, they are processed to ensure they're in the right format for the model to learn from them effectively. Next, the images are used to train the object detection model. Finally, if the system assesses the situation as a medium or high threat, it sends an alert to security guards. This alert includes the image where the weapon was detected and details about the threat level.

The Shenoy *et al.* [1] system initiates surveillance and processes the input from two sources:

a live webcam and test videos, Figure 2.10. For the live webcam input, the system employs face detection technology to locate human faces within the video frames. If a face is detected, the next step is face recognition, where the system compares the detected face against a database of known criminal records. Simultaneously, the system also runs object detection algorithms to identify potential weapons in the scene. The input from test videos, on the other hand, is fed into a CNN-GRU model. When either the live feed or the test video analysis results in the identification of a criminal, the detection of a weapon or suspicious behavior, the system converges these findings to determine if there's a crime happening. If a crime is detected, the system generates an alert and sends a notification to the concerned authorities.

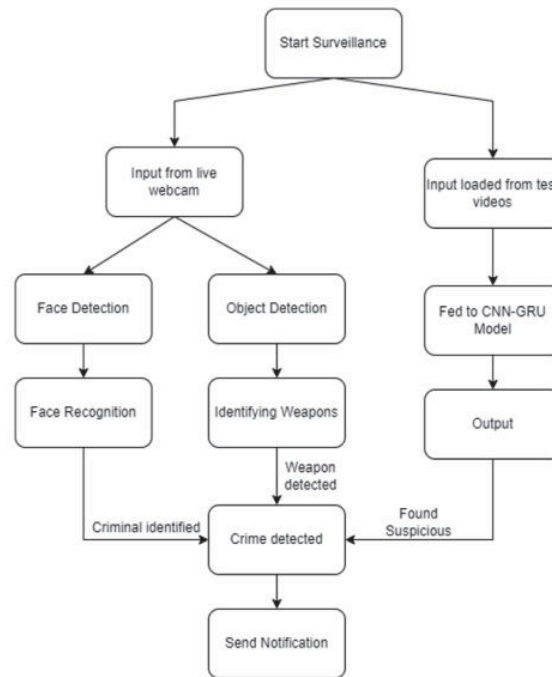


Figure 2.10: Shenoy *et al.* [1] System Architecture

One of the standout features of Gawade *et al.* [2] architecture, Figure 2.11, is its integration of an alarm system that activates upon successful weapon detection, thereby ensuring immediate alert to security personnel. This aspect of the model not only enhances its practical

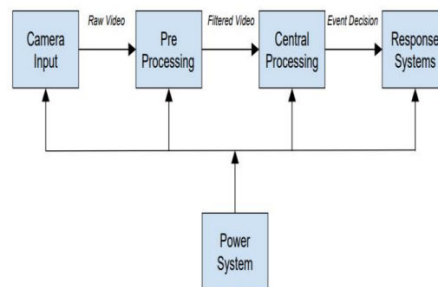


Figure 2.11: Gawade *et al.* [2] Architecture Diagram

application but also significantly contributes to the safety and security in public spaces like

malls, airports, and railway stations.

Al-Mousa *et al.* [3] system's operation begins with a Raspberry Pi Camera, which serves as the image capture device, feeding live images into the processing unit, a Raspberry Pi. Once the images are pre-processed, they are fed into a pre-trained Deep Neural Network (DNN) weapon detection model. Upon successful detection of a weapon, the system activates an alert mechanism. Finally, the system sends a notification along with the images of the detected weapon to a smartphone, as represented in diagram 2.12.

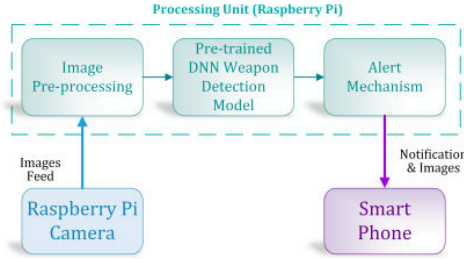


Figure 2.12: Al-Mousa *et al.* [3] System Diagram

2.3.4 Comparative Analysis of Weapon Detection Technologies

Bhatti *et al.* [5] emphasize the importance of security for economic growth and attracting investors and tourists. The study addresses the difficulty of detecting weapons in real-time, considering problems like different angles, blocked views, and hidden weapons. A dataset was adapted by the researchers from diverse sources, including original photos, online repositories, and even film databases. Their exploration spanned several algorithms, like VGG16, Inception variants, and YOLO iterations. Through rigorous testing prioritizing precision and recall over mere accuracy, YOLOv4 emerged as the frontrunner, achieving a F1-score of 0.91 and a mAP of 0.92. To illustrate the methodology behind the advanced surveillance techniques discussed, Figure 2.13 provides a schematic overview of the relationship between object recognition, including classification and localization, and object detection.

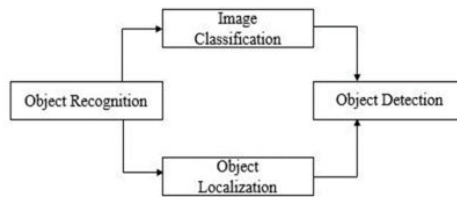


Figure 2.13: Object Recognition to Detection Hierarchy [5]

Rehman and Fahad [31] point out the concern of incidents involving firearms and knives due to insufficient security checks. Despite CCTVs have become prevalent, their constant surveillance demands often surpass human monitoring capabilities. The paper introduces an automated weapon detection system, which employs the YOLOv5 deep learning model, adapted to a carefully selected dataset. The obtained results achieved a F1 score of 0.95 when applied to CCTV footage.

In their study, Thangaraj *et al.* [33], address the challenge of detecting concealed weapons in public spaces, despite widespread CCTV surveillance. They point out the limitations of human monitoring in effectively analyzing extensive footage. To overcome this, they develop an enhanced weapon detection system using the YOLOv5 deep learning framework, specifically trained on a dataset of 1496 images. This system excels in identifying handguns in various conditions, achieving a mAP of 0.92. Additionally, their comparative analysis with YOLOv3, as shown in Table 2.3, indicates that YOLOv5 not only improves accuracy in handgun detection but also operates faster and with lower computational demands.

Table 2.3: Models Performance Comparison [33]

Model	mAP (%)	Detection Speed/Image	Frames per second	Parameters No. (M)	Model Size (MB)
YOLOv3	78.90	0.054	13	7.5	236
YOLOv5	91.76	0.023	24	62	14

Moran *et al.* [6] refers in smart cities context the need for a advanced surveillance applied to urban security. The authors present an approach, combining super-resolution techniques with the YOLOv4 deep neural network, to detect knives in complex images, a task made difficult by variables like shape, size, and lighting. Table 2.4 shows the results of their models, where M3 achieved the best overall performance. Models without transfer learning (M1 and M3) benefitted from SRGAN preprocessing, while those with transfer learning (M2 and M4) performed worse. Model M1 had the highest average IoU, and M3 excelled in Precision, Recall, and F1-score.

Table 2.4: Moran *et al.* [6] Model General Results

Model	TP	FP	FN	IoU (mean)	Precision	Recall	F1-Score
M1	2057	143	98	0.776	0.935	0.955	0.945
M2	1071	774	1084	0.269	0.580	0.497	0.535
M3	2064	32	91	0.756	0.985	0.958	0.971
M4	727	1211	1428	0.141	0.375	0.337	0.355

Safa and M [37], address the challenge of escalating urban crimes involving firearms and sharp objects in their study. They note the inadequacy of manual monitoring of the extensive data generated by omnipresent CCTVs in modern cities. To bridge this gap, they have innovated using the YOLOv8 deep learning framework, tailored to a specialized dataset including images of pistols, knives, and screwdrivers. The system achieved 0.93 accuracy rate on CCTV footage.

Gawade *et al.* [2] research tackles the increasingly alarming scenario of weapon-related threats in crowded public spaces like banks, airports, and railway stations. Recognizing that while the deployment of CCTV cameras has seen a global surge, the sheer volume of footage can overwhelm human operators. This paper introduces a weapon detection system that uses the power of deep learning, specifically through the use of CNN. Trained on a dataset of 10014 images, the model adeptly identifies knives, small guns, and long guns, achieving an

accuracy rate of approximately 0.85.

In the security system presented by Al-Mousa *et al.* [3], a key component of the image processing workflow involves the pre-processing of CCTV feed images to enhance the performance of the deep learning model, Figure 2.14. The system operates by periodically capturing images from CCTV feeds and analyzing them with a CNN, enabling swift identification of potential threats. A distinctive feature of this system is the immediate notification of security personnel via a mobile app, including an image of the detected threat. The system achieves an accuracy rate of 0.92 and can detect threats within 1.6 seconds.

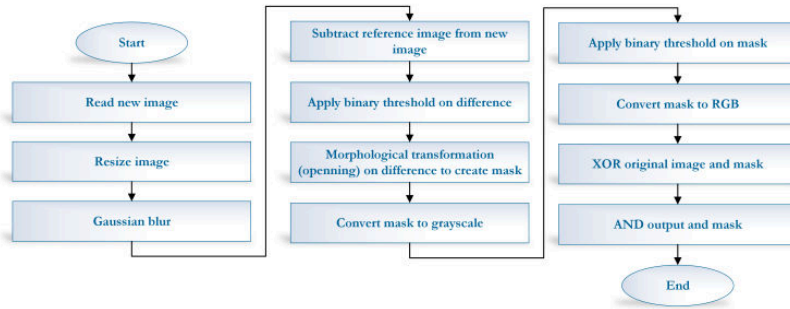


Figure 2.14: Al-Mousa *et al.* [3] Image Pre-processing Flowchart

Hnoohom *et al.* [4] address the limitations of CCTV systems, particularly their dependence on manual monitoring by police personnel, by introducing an automated weapon detection system, Figure 2.15. Utilizing two public datasets, ARMAS and IMFDB [32], their research tests various object detection techniques, including SSD MobileNet-V1, EfficientDet-D0, and Faster-RCNN Inception Resnet-V2. The study highlights the effectiveness of Faster-RCNN Inception V2 with the ARMAS dataset, achieving a mAP of 0.540, and AP scores of 0.793 at 0.5 IoU and 0.627 at 0.75 IoU.

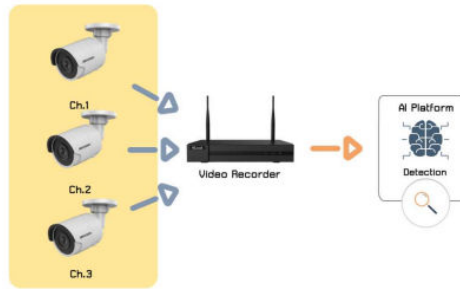


Figure 2.15: Hnoohom *et al.* [4] Weapon Detection Process

Shenoy *et al.* [1] emphasize the alarming rise in criminal activities in urban spaces, despite the ubiquity of CCTV systems in public areas. Recognizing the limitations of human supervision for these vast surveillance networks, they introduce an intelligent crime detection mechanism. Utilizing the SSD Mobilenet architecture fine-tuned on the DaSCI Weapon Dataset [35], their solution identifies a broad spectrum of weapons, yielding an accuracy rate of 0.81. Moreover, with the integration of the GRU-based behavior analysis model, the system achieved an accuracy of 0.96 in detecting suspicious actions.

Table 2.5 provides a comparative summary of the weapon detection models used by the authors. YOLO models, particularly YOLOv5, demonstrate high performance with an F1 score up to 0.95, indicating strong precision and recall. The table also shows that while traditional CNN models are effective, with accuracy scores around 0.92, are generally outperformed by the YOLO series. Faster-RCNN and SSD models show varied results with accuracies ranging from 0.81 to 0.85 and mAP scores as low as 0.54, suggesting they may be less consistent across different datasets. Overall, newer YOLO models seem to offer a good balance of accuracy and efficiency for object detection tasks.

Table 2.5: Authors models and results

Authors	Model	Results
Rehman and Fahad [31]	Yolov5	F1 Score 0.95
Bhatti <i>et al.</i> [5]	Yolov4	F1 Score 0.91, mAP 0.92
Thangaraj <i>et al.</i> [33]	Yolov3, Yolov5	mAP 0.92
Moran <i>et al.</i> [6]	Yolov4	Precision 0.935, Recall 0.955, F1 Score 0.945
Safa and M [37]	Yolov8	Accuracy 0.93
Al-Mousa <i>et al.</i> [3]	CNN	Accuracy 0.92
Gawade <i>et al.</i> [2]	CNN	Accuracy 0.85
Hnoohom <i>et al.</i> [4]	Faster R-CNN	mAP 0.54, AP 0.79 (0.5 IoU), AP 0.63 (0.75 IoU)
Shenoy <i>et al.</i> [1]	SSD	Accuracy 0.81

Figure 2.16 presents the distribution of different models utilized in previous weapon detection studies, with YOLO models being the predominant choice at 55.6%, featured in over half of the articles.

Meanwhile, Figure 2.17 displays the evaluation metrics that researchers selected for assessing weapon detection models. The Accuracy Rate is the leading metric, employed in 33.3% of the studies, followed by the F1 Score and mAP, which are used in 25% of the cases. The Precision/Recall and IoU metrics are less commonly used, at 8.3%.

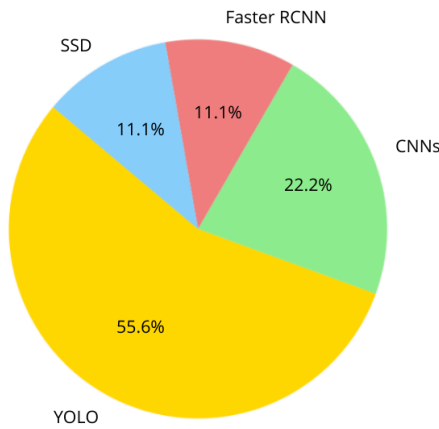


Figure 2.16: Distribution of Models Used in Studies

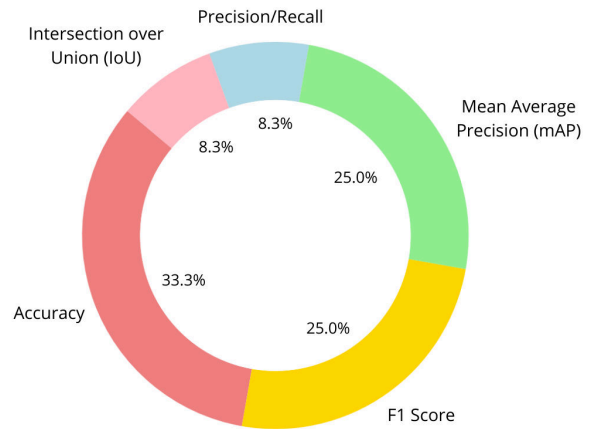


Figure 2.17: Distribution of Metrics Used in Studies

Architecture, System Design and Requirements

This chapter explores the primary architecture that supports the application, outlining its core components and interactions. Also, it covers the basic use cases, along with a comprehensive examination of both functional and non-functional requirements essential for the system's optimal performance.

3.1 BASIC USE CASES

The diagram 3.1 outlines the main use cases of the application, structured around the user interaction flow.

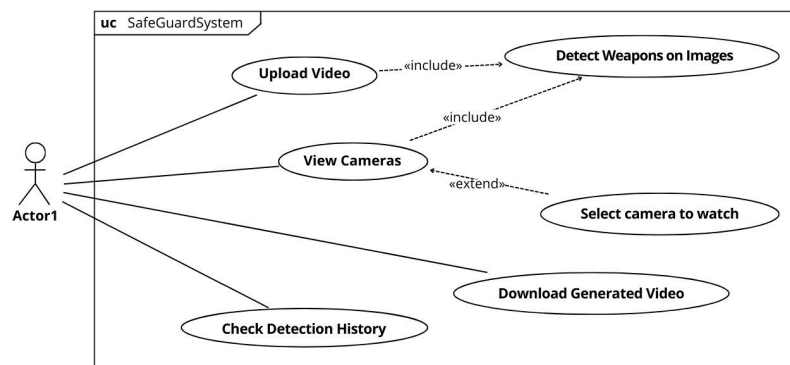


Figure 3.1: System Basic Use Cases [38]

The main actor of the system is the police officer, who is responsible for monitoring and managing the system's surveillance capabilities

Users have the ability to view live camera feeds, which provides real-time surveillance. The system must ensure that all cameras are operational and accessible. Users can select specific cameras to focus on, allowing for detailed monitoring of particular areas.

The upload video functionality allows a user to upload video content to the system. Once a video is uploaded, the system automatically processes it to detect any weapons shown in the images. The user must be authenticated and authorized to upload videos. After the video is processed, the results are stored in the system's database.

After videos have been processed, users can download these videos. This function is essential for users who need to keep offline records or further analyze video content. The system handles the retrieval and download of these videos, ensuring that users can access them conveniently and securely.

Check detection history use case allows users to review and audit the detection activities that have occurred. Users with the appropriate permissions can access detailed logs that document the specifics of detected threats, and when they were detected, providing a comprehensive audit trail.

3.2 FUNCTIONAL REQUIREMENTS

The main goal is to design and implement a real-time weapon detection system capable of analyzing surveillance footage and triggering immediate alerts. A prototype showcasing this functionality was developed. However, during testing and demonstrations, limitations became evident. Sometimes video streams wouldn't load consistently, and alerts were slow to generate. Feedback from security personnel emphasized the need for a more user-friendly interface, allowing for quick navigation between cameras and detailed information about detections.

These critical insights guided the development of the core functionalities outlined below:

1. Video Stream Handling

- The system shall accept video feeds from various sources, continuously monitoring the video streams and analyze frames for weapon detection;

2. Detection and Alert Mechanism

- The system shall generate real-time alerts upon detecting a weapon;
- The system shall classify and log each detection event, including time, location, and type of weapon detected;

3. Video Upload for Detection

- The system shall allow users to upload pre-recorded video files (.mp4) for weapon detection analysis;

4. Detection History

- The system shall store video footage and detection logs securely;
- The system shall allow authorized personnel to access and retrieve stored video footage and logs for review and analysis;

3.3 HIGH LEVEL ARCHITECTURE

Figure 3.2 illustrates the architecture for the proposed system to automatically detect weapons.

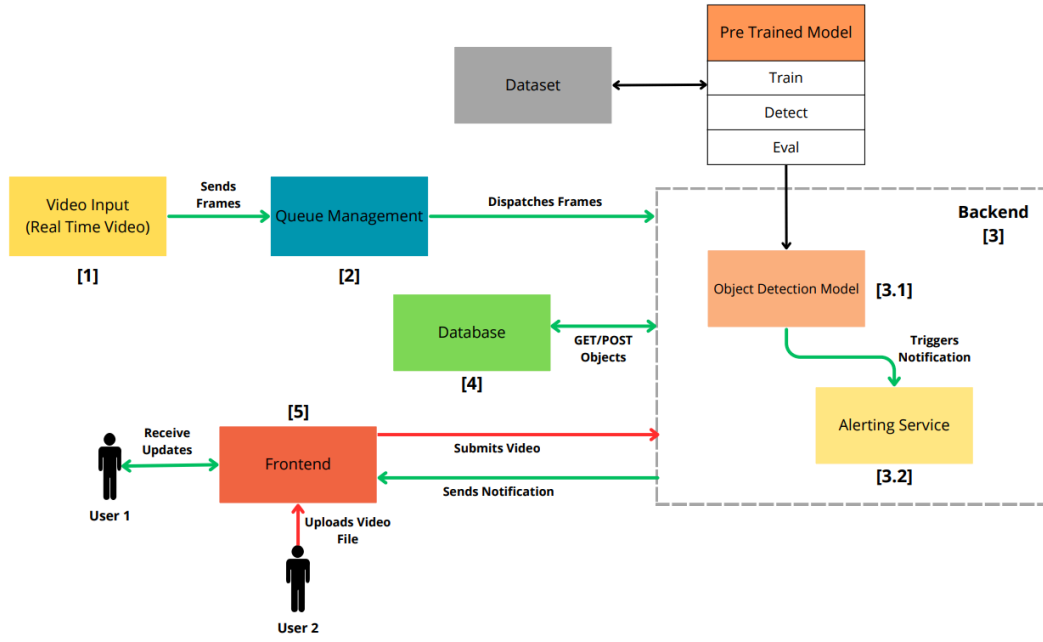


Figure 3.2: High Level Architecture Proposal

The module Video Input (Real-Time Video) [1] aims to feed the system with real-time video using the various CCTV cameras that are spread across various locations. The video is presented to the system frame by frame, and analyzed in the following modules.

The Queue Management component [2] deals with incoming data from previous module, and regulates the sequence flow. This is a management task, from the balance of received frames and sending to further analysis. Therefore, the following items enumerates the tasks that are associated to this module:

- Load Balancing: manages the influx of video frames, particularly crucial during periods of high input. Without this, the backend could become overwhelmed, potentially leading to delays or processing failures;
- Prioritization: allows the prioritization of video frames or streams, especially those from high-risk areas;
- Scalability: enables efficient scaling without necessitating significant modifications to the backend processing infrastructure.

The backend [3] serves as the system's "central processing unit". It is composed by a set of submodules:

- Object Detection Model [3.1]: designed to identify objects of interest, primarily focusing on potentially dangerous items like firearms and knives. It operates with a pre-trained model that was trained on a dataset, which has been finely tuned to pinpoint threats with precision.
- Alerting Service [3.2]: upon detection of a hazardous object, it promptly generates notifications/alerts. These alerts are then swiftly relayed to the frontend, thereby enabling law enforcement officials to receive real-time updates and respond accordingly.

The database [4] stores the critical data generated by the system, like que video metadata, logs of detected objects, model-related data, and other information that supports the system.

Finally, the Frontend [5] enables the interface between system and end users (such as law enforcement officers). Tasks like the uploading of video files for analysis can be performed throw this interface. Moreover, it serves as a dynamic platform for receiving timely updates and alerts regarding any detected objects, thereby streamlining the communication process and enhancing the efficiency of law enforcement responses.

3.4 NON-FUNCTIONAL REQUIREMENTS

This section outlines the non-functional requirements essential for the system’s successful implementation and operation. Non-functional requirements define the system’s operational attributes, which are crucial to ensuring the system’s efficiency and effectiveness in real-world scenarios.

The non-functional requirements can be categorized as follows:

1. Performance
 - Latency: the system must process video frames with minimal delay;
 - Throughput: the system should handle multiple video streams simultaneously without significant performance degradation;
 - Accuracy: high detection accuracy is essential to minimize false positives and false negatives;
2. Reliability and Availability
 - Uptime: the system must maintain high availability, ensuring continuous surveillance and monitoring without interruptions;
 - Fault Tolerance: the system should be resilient to hardware and software failures;
3. Security
 - Data Protection: the system must include secure data transmission and storage using encryption methods to prevent unauthorized access;
 - Access Control: strict access control mechanisms should be implemented to restrict system usage to authorized personnel only;
4. Easy Navigation
 - The interface shall allow users to navigate easily between different video feeds and alert logs;
5. Usability
 - User Interface: the system must provide an intuitive and user-friendly interface for operators, including visual indicators for detected threats and straightforward navigation between video feeds;
6. Performance Monitoring
 - The system shall continuously monitor its performance, including latency and throughput metrics, and provide administrators with tools to analyze and optimize system performance;
 - The system shall generate reports on detection accuracy and system performance for regular review.

YOLO Model for Video Analysis

This chapter explores the evolution of the YOLO family of models, with a particular focus on the capabilities of YOLOv5. Additionally, it outlines the process of dataset creation, highlighting the steps taken to ensure high-quality data for training and evaluating the model in video analysis applications.

4.1 YOLO VERSIONS OVERVIEW

YOLO, which stands for "You Only Look Once" is a real-time object detection system that addresses problems as a regression task. Developed by Joseph Redmon and his team, YOLO employs a single neural network optimized end-to-end for detection performance, enabling rapid and accurate object detection [39].

The Figure 4.1 describes the timeline of YOLO versions, highlighting the progression and key developments from its inception to the latest versions. The timeline begins with YOLOv1 introduced in 2015 and extends to YOLOv8 and YOLO-NAS in 2023.

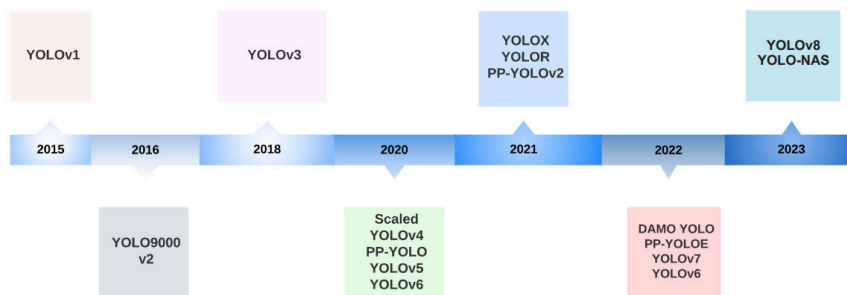


Figure 4.1: YOLO Versions Timeline [40]

Lavanya and Pande [41] work shows a summary of the key improvements and characteristics of each YOLO version:

- YOLOv1 (2015): first real-time object detection system with a single convolutional network pass;

- YOLOv2 (2017): this version included higher resolution and anchor boxes. It contains 5 max-pooling layers, implemented with Darknet, and has 19 convolutional layers;
- YOLOv3 (2018): focused on improving performance on smaller objects, it increased the number of layers to 106;
- YOLOv4 (2020): known for optimal speed and accuracy, it features 53 convolutional layers of specific sizes;
- YOLOv5 (2020): introduced mosaic augmentation and consists of 80 classes of 3 parts;
- YOLOv6 (2021): includes architectural improvements, although the total number of layers is not fixed;
- YOLOv7 (2021): designed for faster inference and greater accuracy, the total number of layers is also not fixed;
- YOLOv8 (2023): offers improved adaptive training, customizable architecture, and advanced data augmentation, with the total number of layers not fixed.

YOLOv5 was chosen for its balance of speed and accuracy, building on the strengths of previous YOLO versions to enhance detection performance. Its stability and maturity have been proven through extensive real-world testing and a strong support ecosystem, making it a reliable choice. Besides, prior knowledge and experience with this version facilitated smoother integration and application. While newer versions may offer specific improvements, they do not have the same level of reliability and resource availability as YOLOv5, making it the preferred option for reliable and efficient object detection tasks. A good point to consider for the future is to evaluate the integration and use of a newer version. Testing it and applying it in the application could bring better results.

4.2 YOLOv5 MODEL OVERVIEW

The YOLOv5 architecture, as depicted in the Figure 4.2, is organized into three primary sections: the Backbone, the Neck, and the Head.

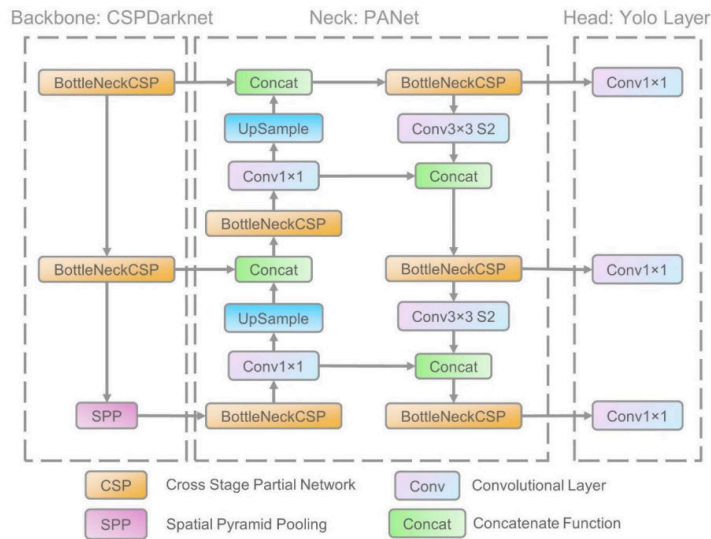


Figure 4.2: YOLOv5 Architecture [42]

The backbone of the network, CSPDarknet, is essential for the initial processing and feature extraction from raw input data. It uses multiple BottleNeckCSP blocks, which are crucial to the Cross Stage Partial Network (CSPNet) [42].

CSPNet is designed to address a common issue in large-scale neural networks - the redundancy of gradient information. It occurs when the same or similar gradient information is repeatedly used to update the network parameters, even though it does not contribute new or valuable insights to the learning process. Gradients are essential for adjusting the network during training, providing the necessary information to update the network's parameters, however, when redundant, they can slow the process.

CSPNet, represented in the Figure 4.3, addresses this issue by splitting the feature map of layers into parts at various stages within the network. Using this strategy, split and merge, reduces the redundancy of the gradient flow through the network, leading to more efficient learning processes.

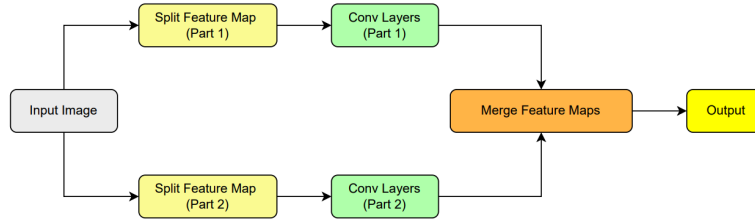


Figure 4.3: CSPNet - Cross Stage Partial Network

Feature extraction in this context involves using convolutional, pooling, and bottleneck layers to transform the raw input image into a set of feature maps. These maps encode essential visual information such as edges, textures, and other significant patterns that are pertinent for detecting objects.

Following the backbone, the Neck of the YOLOv5 architecture features the Path Aggregation Network (PANet). PANet enhances the Feature Pyramid Network (FPN) with a strong bottom-up pathway, which improves the propagation and utilization of lower-level features across the network. This structure helps in better localization and detection accuracy, particularly for varied object sizes.

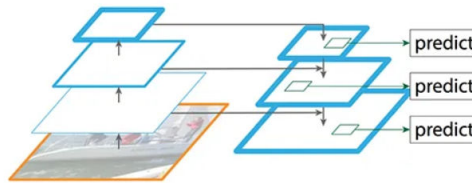


Figure 4.4: Feature Pyramid Network

FPN, exemplified in Figure 4.4, is crucial for constructing a pyramid of feature maps at multiple scales, allowing the network to detect objects effectively across different sizes. It does this by integrating semantic information from deep, coarse resolution feature maps with finer-resolution features from earlier layers through lateral connections and top-down

pathways. This combination ensures that both high-level and low-level features contribute to the final detections.

The final section is the YOLO Layer, which forms the detection head of the network. This layer processes the enriched feature maps to predict bounding boxes, object classes, and confidence scores.

4.3 DATASET PROPOSED

The main purpose of our work is to detect firearms and knives (objects), however different tasks must be performed: select the dataset, train the dataset (with different configurations) and test the model under new test data. Therefore, the outcome of these tasks is a model to be used as part of our system. Initially, to select our dataset we conduct a search based on the following parameters:

- datasets annotated that contains images related to our domain;
- available for download;
- datasets used on scientific papers (not mandatory).

The datasets discovered that fulfill these parameters are:

- DaSCI Dataset [35]: includes images of exposed weapons, specifically knives and handguns, along with various random objects. Originally containing six different classes, it was restructured to include only two categories: 'knife' and 'weapons';
- Mock Attack Dataset [43]: images captured from three strategically positioned surveillance cameras in corridors and an entrance, this dataset offers multiple viewing angles of scenarios where individuals are wielding weapons. The images were annotated at a rate of two frames per second;
- Action Recognition and Object Detection Dataset [44]: several videos categorized into three types: no guns, handguns, and machine guns, this dataset required significant modifications. The annotations were initially in JSON format and had to be converted into .txt format compatible with YOLOv5¹. Additionally, video frames were extracted to form a static image dataset suitable for the training process.

Next, we performed the train and test. Figure 4.5 describe the pipeline for training and testing the selected dataset.

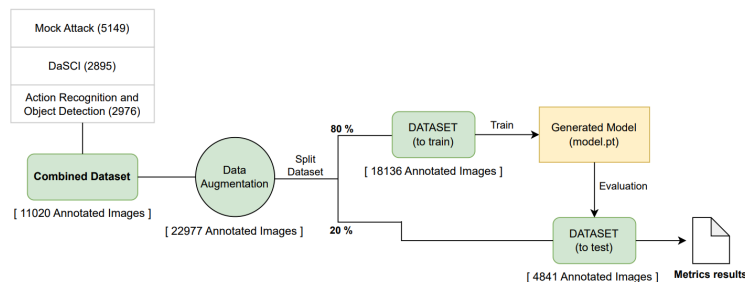


Figure 4.5: Dataset Creation Steps

¹https://github.com/pedromonteiro01/yolo-utils/blob/main/json_to_txt_annotations.py

First, the dataset is submitted into data augmentation task divided into two subtasks: in the first task, images were randomly blurred and rotated to produce some variability; in a second task, images underwent brightness adjustments, resulting in sets of images that were either darker or lighter ².

The dataset resulted has a total of 22977 images, containing more images of handguns than knives. These images were divided into 80% to train, and 20% to test. The model generation were submitted to a fine tuning to achieve the best metrics results.

²<https://github.com/pedromonteiro01/yolo-utils/blob/main/augment.py>

SafeGuard System

In this chapter, the SafeGuard implementation architecture is described, detailing each component and the technology stack utilized, providing a comprehensive overview of how all parts of the system are integrated and work together.

5.1 ARCHITECTURE OVERVIEW

Building this weapon detection system architecture, described in Figure 5.1, raises several challenges, especially integrating a trained model that detects weapons in real time environment, and its large-scale deployment.

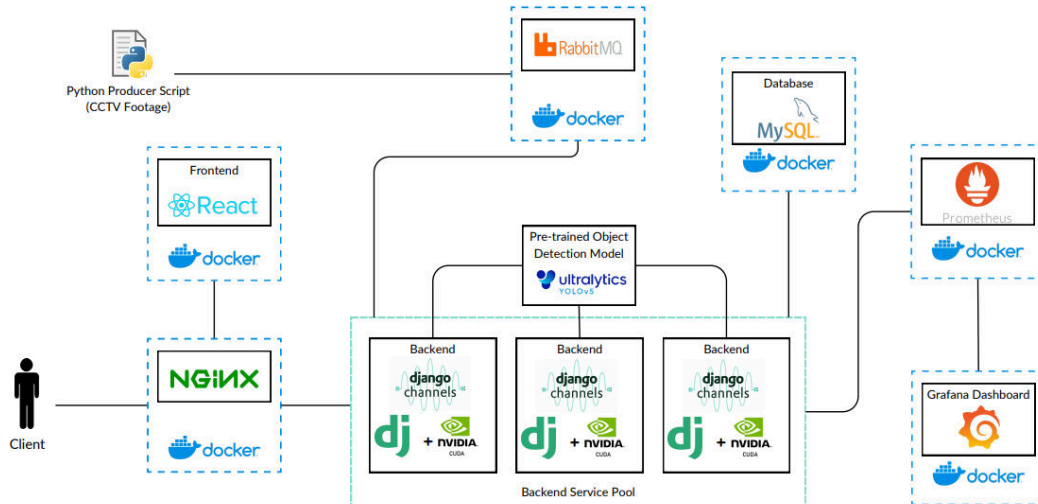


Figure 5.1: Implementation Architecture Overview

First, we developed a Python 3.10 script that simulates the CCTV footage, acting as a producer by sending video frames to the next stage (Queue Management), reproducing real-time surveillance. Because we are dealing with various cameras with different video streams, the Queue Management was built under the RabbitMQ [45], as message broker,

which allows to organize video streams into queues specific to each user, containing the video streams from their respective cameras. This structure not only helps in managing the video data efficiently but also ensures that each user’s data is handled securely and separately.

The backend of the system is built using Django 5.0.6 [46], chosen for its rapid development and maintainability, and employs GPU/Compute Unified Device Architecture (CUDA) technology to enhance the running of detection model inference, a pre-trained YOLOv5 object detection model, fine-tuned with multiple hyperparameters. This component consumes the video data queued in RabbitMQ, processing each frame. Acting as the system’s backbone, it coordinates both video processing and user management. Additionally, it supports user authentication and CRUD (Create, Read, Update, Delete) operations, interacting with a MySQL [47] database essential for storing and managing user data and detection logs.

An essential feature of the system is its ability to detect weapons in the video frames. Upon detection, the backend uses Django Channels and WebSockets to trigger real-time alerts. These alerts are instantly pushed to the frontend, ensuring immediate user notification, which is crucial for prompt response and intervention in security-sensitive situations.

To manage client requests efficiently, Nginx 1.27.0 [48], known for its speed and efficiency, has been deployed as a load balancer, distributing requests evenly among three backend services. Each backend service shares the pre-trained model, ensuring consistency and reliability in detection results. Nginx’s lightweight design minimizes its resource usage, allowing the system to run smoothly, ensuring high availability and improved performance of the system.

The user interface is built with React 18.0.2 [49], influenced by the familiarity with React for creating user-friendly web apps and also for its flexibility. The interface, designed to be intuitive and user-friendly, provides real-time alerts from the backend, a dedicated page listing all associated cameras, and a historical log of detections. Additionally, there’s an upload page for analyzing past recorded videos.

Furthermore, Prometheus 2.52.0 [50] and Grafana 11.0.0 [51] have been integrated for monitoring and visualization of system performance metrics. This duo provides a clear picture of how the system is running, allowing for proactive maintenance and real-time tracking of system health. Prometheus collects data on various aspects of the system’s performance, and Grafana transforms this data into easy-to-understand charts and graphs.

The technologies will be better explained and analyzed in the following sections.

5.2 FOLDER STRUCTURE

This section outlines the structure of the project directories and files, designed to support a modular and scalable application, Figure 5.2.

The root directory, *project/*, serves as the central hub containing all the subdirectories and configuration files pertinent to the project. It includes:

- Nginx configuration file
- Docker configuration file
- Producer script
- Backend folder

- Frontend folder

The *backend/* directory of the project has the server-side logic, developed using the Django framework. This setup includes all necessary components of a Django application, including models, views, and controllers. These elements handle the business logic and database operations and are located within the *base/* folder. The directory also contains a *requirements.txt* file, crucial for defining all the Python dependencies required for the backend. Additionally, it includes the detection model files.

In contrast, the *frontend/* directory focuses on the client-side elements of the application, utilizing React to build the user interface. This directory contains the main React components and views. It also includes a Dockerfile, which specifies how to build the Docker container for the React application. The *components/* subdirectory holds individual javascript files that define specific parts of the user interface.

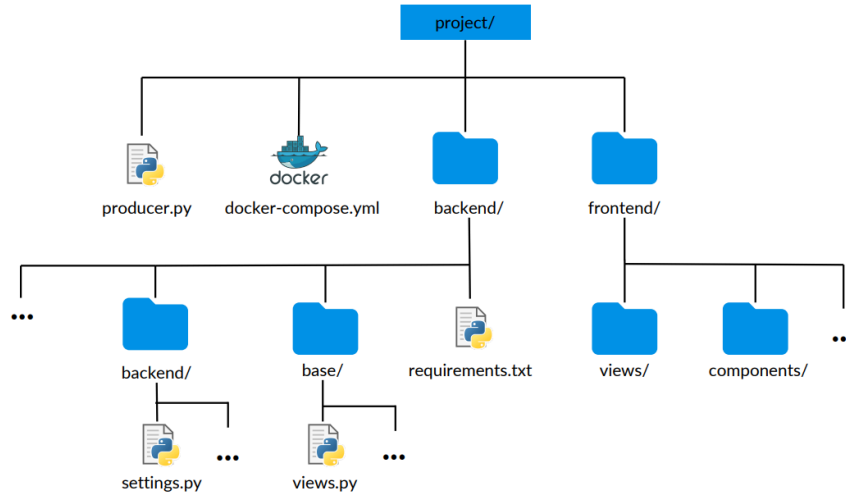


Figure 5.2: Folder Structure Diagram

5.3 VIDEO RETRIEVAL AND MANAGEMENT

5.3.1 RabbitMQ Setup

RabbitMQ is a widely adopted message broker that efficiently manages messaging and streaming, making it an ideal choice for applications that require robust and scalable message handling. It is designed for reliability and flexibility, easily deployed across cloud environments, on-premises, or locally, serving millions of users worldwide [45]. Its versatility and reliability make RabbitMQ a fitting backbone for our Django-React application, where it orchestrates video data streaming from simulated CCTV cameras.

To integrate RabbitMQ into our Django-React application, we employ Docker for containerization, which simplifies the deployment and scalability of services. The configuration of RabbitMQ within a Docker environment is specified in the *docker-compose.yml* file. Below, are outlined the key settings:

- Image:

- *rabbitmq:3-management* image is used, which includes the management plugin for easy monitoring and management through a web-based interface.
- Environment Variables:
 - *RABBITMQ_DEFAULT_USER* sets the default username.
 - *RABBITMQ_DEFAULT_PASS* specifies the default password.
- Ports:
 - 5672 is exposed for RabbitMQ server connections.
 - 15672 is exposed for accessing the RabbitMQ management interface.
- Volumes:
 - a dedicated volume, *rabbitmq-data*, is used to persist RabbitMQ data and configurations, ensuring data durability across container restarts.

5.3.2 Message Queuing and Processing

The message queuing and processing subsystem is crucial for managing the flow of video data from CCTV cameras to the processing backend efficiently, as illustrated in Figure 5.3.

RabbitMQ operates by holding, temporarily, messages in queues until they can be safely processed by consumers. The setup process involves configuring RabbitMQ queues for each CCTV camera source. These queues store video frames that are encoded as Base64 strings and tagged with metadata: *camera_id*, *user_id* and *timestamp*.



Figure 5.3: RabbitMQ Processing [52]

The process initiates with each video file from the CCTV camera being processed frame by frame. The OpenCV library [53] is used to capture video frames, which are then encoded into a JPEG format using base64 to facilitate easy transmission over the network. Each frame is packaged into a JSON object containing relevant metadata and pushed to the designated RabbitMQ queue. The key function *send_frame_to_queue* is responsible for publishing each frame to its respective queue. It constructs a message payload containing the encoded frame and metadata, then sends this message to the RabbitMQ exchange with a specific routing key corresponding to the queue name.

To enhance the throughput and responsiveness of the system, the video processing operation is executed in multiple threads. Each thread handles a different video source, allowing the system to parallelize the workload effectively.

5.4 FRONTEND DEVELOPMENT

5.4.1 Views Implementation

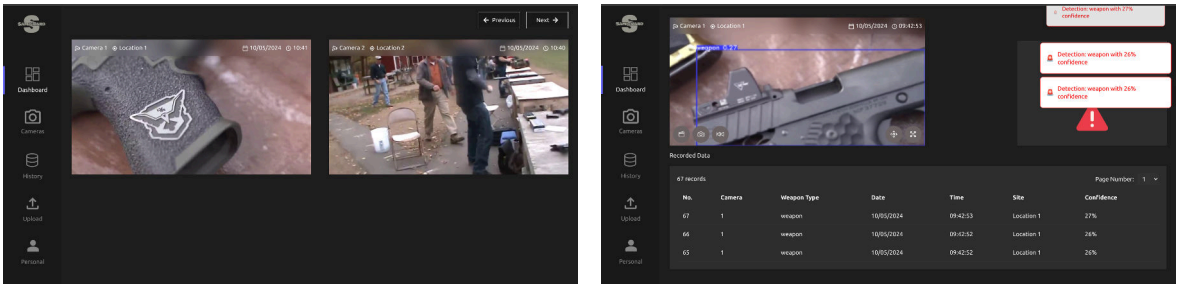
After authentication, user is presented with the first view which lists all the cameras associated with their account, as showed in Figure 5.4a. This ensures that users have organized access to their specific surveillance resources.

As mentioned above, the system uses RabbitMQ, which maintains a queue dedicated to each user. When cameras stream video, RabbitMQ receives these frames and forwards them to the backend. From there, the backend processes these frames and sends them to the frontend via WebSockets.

Upon selecting a camera, the user is directed to a second view where the video footage is streamed live, as illustrated in Figure 5.4b. In parallel, the stream is analyzed using an object detection model. The details of this model, which will be discussed subsequently, include the identification of specific objects of interest, in this instance, weapons.

When a weapon is detected, the system triggers an alert that is immediately relayed to the user through a toaster notification on the frontend interface. This alert specifies the type of weapon detected and the confidence level of the detection, providing information immediately to the user.

Furthermore, the system ensures that all detections are recorded in a database in real time, thus providing a comprehensive and accurate record of events. This ability to retain data indefinitely permits retrospective analysis, which can prove instrumental in both investigative and monitoring activities.



(a) User Listed Cameras (b) Single Camera Analysis

Figure 5.4: Single and Multi Camera Pages

Users may access the aforementioned historical data through a History page, as illustrated in Figure 5.5, where they can view specific instances of detected weapons, including the time of detection and the captured frame.

86 records

No.	Camera	Weapon Type	Date	Time	Site	Confidence	Action
1	1	weapon	2024-05-10	09:43:24.085409	Location 1	56%	🔍
2	1	weapon	2024-05-10	09:43:24.044393	Location 1	60%	🔍
3	1	weapon	2024-05-10	09:43:23.785621	Location 1	66%	🔍
4	1	weapon	2024-05-10	09:43:23.685387	Location 1	64%	🔍
5	1	weapon	2024-05-10	09:43:23.651937	Location 1	60%	🔍
6	1	weapon	2024-05-10	09:43:23.634259	Location 1	65%	🔍
7	1	weapon	2024-05-10	09:43:23.607927	Location 1	66%	🔍
8	1	weapon	2024-05-10	09:43:23.568254	Location 1	50%	🔍
9	1	weapon	2024-05-10	09:43:23.200299	Location 1	42%	🔍
10	1	weapon	2024-05-10	09:43:22.985138	Location 1	53%	🔍
11	1	weapon	2024-05-10	09:43:22.965544	Location 1	46%	🔍
12	1	weapon	2024-05-10	09:43:22.944791	Location 1	52%	🔍
13	1	weapon	2024-05-10	09:43:22.856580	Location 1	26%	🔍

Figure 5.5: Detections History Page

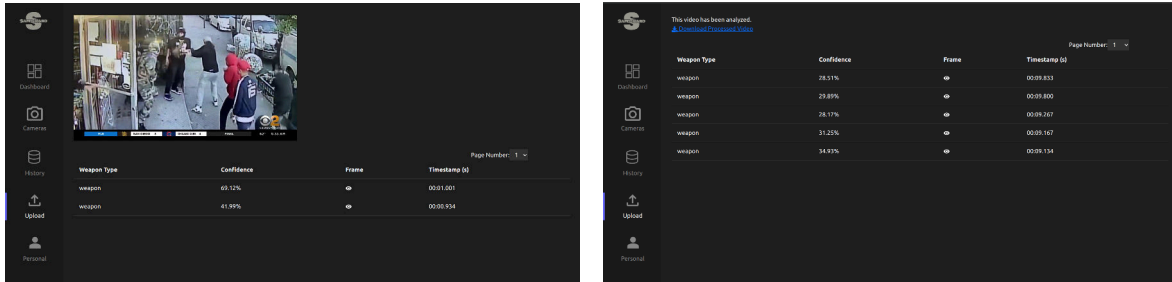
The application includes a section dedicated to the analysis of pre-recorded videos. This

section allows users to select and upload a video file in the .mp4 format.

Once a file is uploaded, it appears in a specific list that displays all of the files they have submitted, providing users quick access to their uploaded content and enabling them to track the status of each video.

Each file in the list is followed by a status indicator. If a video has not yet been analyzed, its status will reflect this, and selecting the video will lead the user to a page where the analysis process begins afresh, as illustrated in Figure 5.6a. Importantly, any previous detections associated with the video are cleared to ensure that the analysis starts from the beginning, providing a clean slate for accurate detection. The analysis must run to the completion of the video for it to be marked with a check, signifying that it has been fully processed.

For videos that have already been analyzed, users can immediately access and download the processed video, 5.6b. The video version presented here includes bounding boxes, drawn around the weapons identified by the system, which visually indicate the location and presence of potential threats within the footage. Alongside the video, a list of detections are provided in a detailed table. This table includes the type of weapon detected, confidence score and the exact timestamp within the video where each weapon appears.



(a) Upload Video Analysis

(b) Download Video after Analysis

Figure 5.6: Upload Video Feature

5.4.2 Integration with Docker

By using Docker, the frontend is encapsulated in a controlled, consistent environment, ensuring that all dependencies and configurations are unified across different development and production stages.

The frontend service is built using a Dockerfile¹ that specifies *node:alpine* as the base image. The service configuration includes essential commands to set up the working environment, install dependencies, and prepare the service to be run with the *yarn start* command. Specifically, the *COPY* commands transfer the necessary project files into the container, and the *RUN* command installs all node modules as specified in *yarn.lock*.

This setup facilitates a more efficient development process, allowing for rapid testing and deployment while maintaining high reliability and compatibility across various environments. Additionally, the direct transfer of project files and dependency management through Docker

¹<https://github.com/pedromonteiro01/cctv-real-time-weapon-detection/blob/main/project/frontend/Dockerfile>

ensures that the application behaves predictably in every run, reducing potential deployment issues.

5.4.3 Testing and Quality Assurance

In frontend development, testing is a critical component that ensures the application is reliable, user-friendly, and robust. Three main types of testing have been implemented: unit tests, integration tests, and End-to-End (E2E) tests.

The unit tests were developed using Jest [54] with the main objective to test individual components in isolation. This approach focused on ensuring the correct functionality of each component, examining internal logic, state management, and behavior under various conditions.

Integration tests were also conducted using Jest. These tests are crucial for assessing the interactions between connected components and services. The focus here was on the data flow and state management across components.

For E2E testing, was used Selenium [55] with the Firefox driver [56] to simulate real user interactions from start to finish. This testing ensures the system behaves as expected in a production-like environment and includes several specific scenarios:

- Login Test: ensured that authentication works correctly and the user gains access to their account (code 1);
- Upload Video File for Detection: tested the upload of a video file for analysis, ensuring the application handles file uploads and data processing accurately;
- Download Analysed Video File: ensures users can download the processed video files, confirming that the files are accessible;
- Get User Cameras List: verify that the application could correctly interact with the backend to display the list of the user's connected cameras.

Below is an excerpt of the login test, code 1.

```
try:
    driver.get('http://localhost:3000/login')
    driver.find_element(By.NAME, "username").send_keys("pmapm@ua.pt")
    driver.find_element(By.NAME, "password").send_keys("password")
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

    time.sleep(5)

    if driver.current_url == 'http://localhost:3000/personal':
        print("Login successful")
    else:
        print("Login failed")
finally:
    driver.quit()
```

Code 1: Login Test.

The login test navigates to the login page, inputs the username "pmapm@ua.pt" and password "password", and submits the form. After waiting for 5 seconds, it checks if the URL

is `http://localhost:3000/personal` to verify if the login was successful. Finally, it closes the web driver.

5.5 BACKEND DEVELOPMENT

5.5.1 Authentication and Security

This section explores the authentication and security mechanisms implemented in the backend. To manage the users of the application, a custom user model was defined, having key attributes such as email, names, and password.

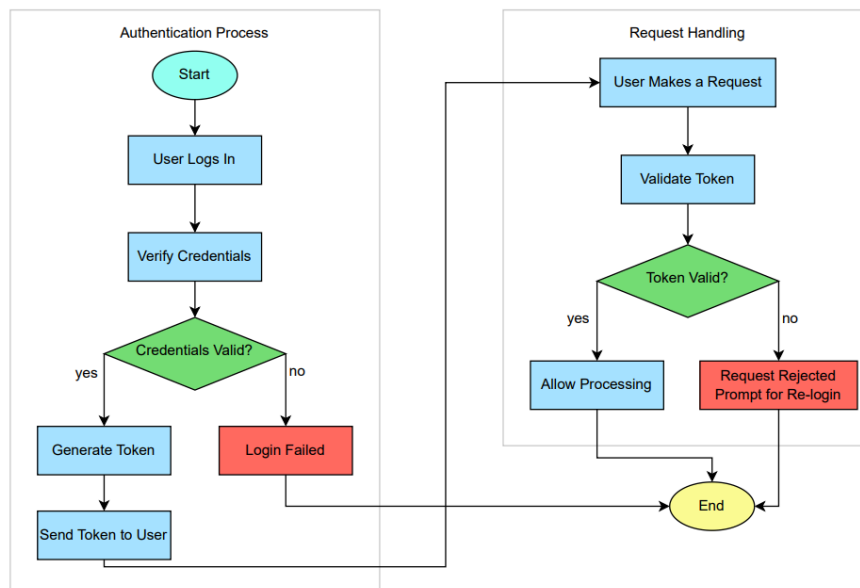


Figure 5.7: Authentication Flowchart

Following the flowchart 5.7, when a user successfully logs into the system using their credentials (email and password), the backend verifies these credentials against the stored data. If the credentials are correct, the server generates a token. This token acts as a stand-in for the user's credentials during future requests, thus avoiding the need to resend the password over the network, which could be intercepted. Each token is unique and tied specifically to one user.

Once the token is generated and sent back to the user's client, a web browser, it is stored locally, in local storage. Every subsequent request will include this token in the request headers, to tell the server which user is making the request.

Upon receiving a request that includes a token, the server first validates the token. This validation process checks if the token is valid, has not expired, and is linked to an actual user account. If any of these checks fail, the server will reject the request.

If the token is valid, the system allows to process the request coming from the user.

For added security, tokens are configured to expire after a certain period. This limits the time an attacker has to use a stolen token. When a token expires, the user must log in again, and a new token is generated.

5.5.2 Client-Server Interaction Flows

The system interactions between the client and server, described in Figure 5.8, are divided into two main processes, each supporting a different aspect of video management and surveillance.

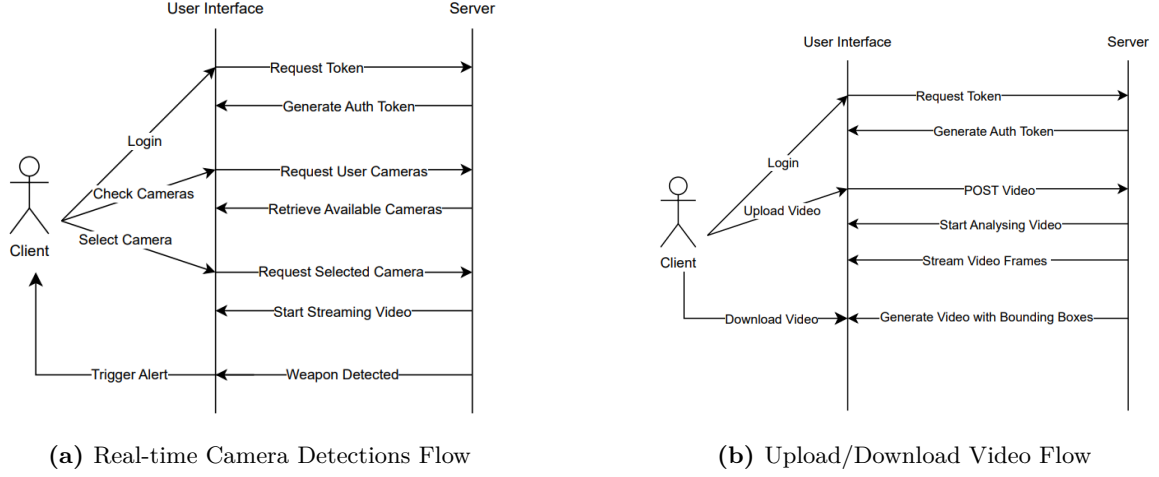


Figure 5.8: Basic Client-Server Interaction Flows

The 5.8a flow begins with the client logging into the system, establishing a secure session through user authentication. Following successful login, the client requests an authentication token, which the server generates and sends back. This token is crucial for validating the subsequent requests by the client under the secure session.

Once authenticated, the client requests access to the cameras associated with their account. The server responds by retrieving and sending a list of available cameras. The user selects a camera from this list, and the client sends a request to access the selected camera. The server, upon receiving this request, starts streaming video from the chosen camera directly to the client.

During the streaming, if the server's analysis algorithms detect a weapon in the video, it triggers an alert. This alert is then sent to the client, notifying them of the potential threat.

The second flow, 5.8b, also starts with the client logging in and obtaining an authentication token, ensuring a secure environment for data exchange. After login, the client uploads a video file for analysis, using a POST request. This video is selected from the user's local device and is sent to the server.

Upon receiving the video, the server initiates the analysis using a pre-trained object detection algorithm.

After completing the analysis, the server generates a new version of the video, which is the output of the model, featuring bounding boxes around the detected objects. This annotated video is then made available for the client to download.

5.5.3 Data Models

In Django, models play a crucial role by serving as the structural foundation of the database. These models, represented in Figure 5.9, enable the seamless translation of database

schemas into Django objects, facilitating CRUD (Create, Read, Update, Delete) operations through Python code, bypassing the need for direct SQL interaction.

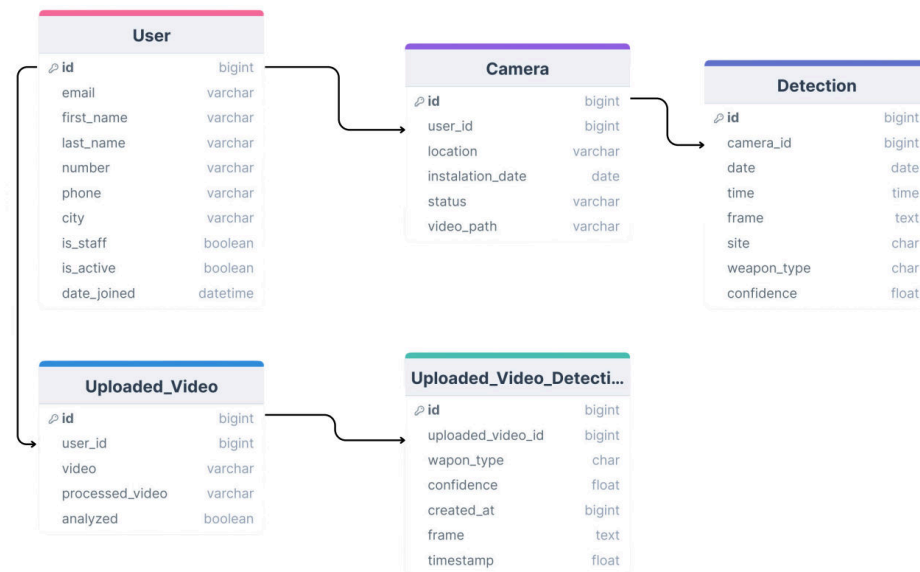


Figure 5.9: Database Schema

The User model at the core of this schema stores not only the credentials that officers use to access the application but also their personal information such as the name, contact number, and city of residence. This model also includes access control attributes, such as staff privileges, for managing permissions within the application.

Each police officer using the platform can manage multiple cameras. These cameras, detailed in the Camera model, are installed in various locations and are integral to capturing continuous video footage. The Detection model is essential for logging specific events detected by the cameras. It captures critical details such as the date, time, frame of detection, and the type of weapon detected, if any, along with the detection's confidence level. This information is vital not only for real-time alerts but also for historical data analysis, which is essential for post-event investigations.

Furthermore, the application supports an upload feature, allowing officers to upload video files for analysis. The Uploaded Video model maintains a direct association with the user, ensuring clear accountability for each video uploaded. It tracks whether each video has been analyzed, helping in the workflow management of video processing.

The Uploaded Video Detection model focus on detections within the uploaded videos. It records similar details as the Detection model, such as the type of weapon detected, the confidence level of the detection, and the specific frame and time within the video where the detection occurred.

5.5.4 API Endpoints

Views are a crucial component that handle the logic and control flow of applications by responding to requests from users. They interact with models and render appropriate responses, usually in the form of HTML pages, JSON, or XML.

The application is structured around three principal groups of endpoints, each dedicated to a specific aspect of the application’s functionality: user management, detections, and uploaded videos.

The first group of endpoints, Figure 5.10, focuses on user management, facilitating secure and straightforward user interactions with the application. The primary entry point is the login endpoint, where users authenticate by submitting their credentials. Successful authentication returns a session token, crucial for maintaining a secure and personalized session. Once authenticated, users can retrieve their profile information such as name, email, and role within the application.

user		
GET	/user/	user_list ✓ 🔒
POST	/user/login/ User Login	user_login_create ✓ 🔒

Figure 5.10: User Views

Detections play a crucial role in the application, supporting both real-time security monitoring and historical data analysis. The POST /detections/ endpoint, represented in Figure 5.11, is designed for the application to submit new detection records to the database in real time when an incident is detected by a specific camera. On the other hand, the GET method allows the application to retrieve all detection records for all cameras associated with a specific user. This endpoint is particularly helpful for creating a historical overview of detections, which users can access to review and analyze past incidents.

detections		
GET	/detections/ Get All Detections	detections_list ✓ 🔒
POST	/detections/ Post Detections For Specific Camera	detections_create ✓ 🔒
GET	/detections/camera/{camera_id}/ Get Camera-specific Detections	detections_camera_read ✓ 🔒
DELETE	/detections/{video_id}/delete/ Delete Video Detections	detections_delete_delete ✓ 🔒

Figure 5.11: Detections Views

Additionally, the application includes a DELETE endpoint, for the removal of detections associated with a particular video. This feature enables the management of stored detection data.

uploaded_videos		
GET	/uploaded_videos/ Get All Uploaded Videos	uploaded_videos_list ✓ 🔒
POST	/uploaded_videos/upload_video/ Upload Video	uploaded_videos_upload_video_create ✓ 🔒
GET	/uploaded_videos/{video_id}/ Get Uploaded Video Details	uploaded_videos_read ✓ 🔒
GET	/uploaded_videos/{video_id}/detections/ List Video Detections	uploaded_videos_detections_list ✓ 🔒
POST	/uploaded_videos/{video_id}/detections/ Upload Video Detections	uploaded_videos_detections_create ✓ 🔒

Figure 5.12: Uploaded Video Views

The uploaded videos section of the application, represented in Figure 5.12, addresses the need to analyze previously recorded or externally sourced video footage. Users can upload

videos through the upload video endpoint, which processes and stores the videos in the database, linking them to the user's account for secure and easy access.

The application also includes a */metrics/* endpoint, which exposes Prometheus metrics using *django_prometheus*. This endpoint provides essential performance and operational metrics for the application, facilitating monitoring and alerting. These metrics cover a range of data points, such as request rates, response times, and error rates. This feature will be discussed in more detail in the subsequent sections.

5.5.5 Django Consumers

This section discusses the three distinct types of consumers used in the application: *VideoStreamConsumer*, *MultiCameraStreamConsumer*, and *UploadedVideoStreamConsumer*.

The core of these consumers' operations begins with the initialization of the computational environment, specifically using GPU acceleration via CUDA for real-time video analysis.

All consumers initiate their processing by setting up the necessary computational resources to ensure efficient video analysis. This setup involves determining the availability of a GPU and utilizing CUDA (Compute Unified Device Architecture) for accelerated computing. `DataParallel` is used to distribute the model across multiple GPUs if more than one GPU is available.

```
device = "cuda" if torch.cuda.is_available() else "cpu"

# load model and wrap it with DataParallel if multiple GPUs are available
model = torch.hub.load('ultralytics/yolov5', 'custom', path='base/best.pt').to(device)
if torch.cuda.device_count() > 1:
    print(f"Using {torch.cuda.device_count()} GPUs!")
    model = DataParallel(model)
```

Code 2: Cuda Initialization.

The *VideoStreamConsumer* is fitted for handling real-time camera footage. Upon a user's selection of a camera, this consumer connects to RabbitMQ. The primary function of this consumer is to continuously receive and analyze the incoming video stream. It employs the initialized YOLOv5 model to detect weapons in the footage, ensuring immediate response and alerting in potentially threatening situations. To optimize performance, only every 10th frame of the video stream is analyzed, as there are not many differences between consecutive frames, making it unnecessary to analyze each one.

For scenarios where multiple camera feeds need to be monitored simultaneously, the *MultiCameraStreamConsumer* is utilized. This consumer fetches video streams of all selected user cameras from RabbitMQ, managing multiple feeds efficiently. In the application interface, it lists these camera feeds, allowing users to monitor various viewpoints within a single dashboard. Similar to the other consumer, also analyzes only every 10th frame of each video stream.

The *UploadedVideoStreamConsumer* addresses the needs of users who upload pre-recorded videos for analysis. Similar to the other consumers, it streams and analyzes the uploaded video concurrently. This consumer is particularly important for post-event analysis.

5.5.6 Database Integration

MySQL provides a robust, reliable, and efficient storage solution that supports the high demands of real-time data processing and retrieval.

The integration of MySQL within the application infrastructure is facilitated through its deployment inside a Docker container. This approach simplifies the process of setting up and managing the database environment. Encapsulating the MySQL instance in a Docker container, ensures a consistent environment across all stages of development and production, which helps in reducing potential discrepancies that can arise from environment-specific configurations.

To connect the Django application to the MySQL database, the necessary configurations are in the *settings.py* file of the Django project.

In the context of this detection system, data primarily consists of structured information such as timestamps, locations, frame details, and metadata extracted from surveillance footage. Given the relational nature of this data, it aligns well with the tabular format inherent to SQL databases. Utilizing MySQL enables efficient storage, querying, and manipulation of this data, thereby enhancing the overall performance of the detection system.

5.5.7 YOLO Model Integration

The integration of the pretrained YOLO model into the backend for video analysis involved several key steps:

- **Model Pre-training:** the YOLO model is first pretrained on the custom dataset to detect objects, as detailed in the section 4.3;
- **Environment Setup:** the necessary packages and dependencies are installed;
- **Model Integration:** pretrained YOLO model's weight file was incorporated into the Django app;
- **Video Processing:** within the Django app, views are set up to handle video processing. When a camera is selected, the video feed is processed in real-time using the YOLO model to detect weapons.

5.6 WEB SERVER CONFIGURATION

5.6.1 Nginx Setup and Load Balancing

Load balancing is crucial for maintaining optimal performance, especially under high traffic conditions, as it ensures consistent response times and maximizes resource utilization. Nginx functions as a high-performance web server and reverse proxy server within the system, efficiently managing incoming HTTP and WebSocket requests. It enhances the system's reliability and scalability by distributing client requests to various backend services. This setup ensures load balancing and efficient resource utilization.

Nginx is deployed using docker-compose with the *nginx:latest* image, ensuring that Nginx uses the specified configuration file (*nginx.conf*).

Load balancing consists of distributing network or application traffic across multiple servers to ensure no single server becomes overwhelmed. This technique enhances the availability and reliability of applications by preventing server overloads and ensuring that client requests are efficiently managed.

Nginx employs load balancing through defined upstream server pools. The *upstream* directive specifies a group of backend servers to distribute requests. Two upstream pools are configured: *backend_api* for API requests and *backend_ws* for websocket connections, as shown in listing 3.

```
# Define upstream server pool for API
upstream backend_api {
    least_conn;
    server 172.17.0.1:8000;
    server 172.17.0.1:8001;
    server 172.17.0.1:8002;
}

# Define upstream server pool for WebSocket connections
upstream backend_ws {
    server 172.17.0.1:8000;
    server 172.17.0.1:8001;
    server 172.17.0.1:8002;
}
```

Code 3: Nginx Configuration Setup

For *backend_api*, Nginx uses the *least_conn* load balancing method. This method directs incoming requests to the server with the fewest active connections, ensuring an even load distribution across all servers in the pool.

The configuration for WebSocket connections under *backend_ws* also distributes requests among the same set of backend servers, maintaining efficient handling of persistent connections.

Nginx listens on port 8080 and routes requests based on URL paths. For the root location, requests to `/` are proxied to a frontend service on port 3000, including headers to forward client details. Requests to `/api/` are forwarded to the backend API servers with headers to maintain client information and a 100MB limit for uploads. For websocket connections, requests to `/ws/` are directed to the backend WebSocket servers with the necessary headers for WebSocket connections.

5.7 MONITORING AND VISUALIZATION

5.7.1 Metrics Collection with Prometheus

Prometheus is an open-source systems monitoring and alerting toolkit designed primarily for reliability and scalability. It works by scraping metrics from applications, storing this data in a time-series database, and allowing powerful queries and visualizations through its flexible query language.

It collects metrics data through a "scraping" process. This involves periodically querying an endpoint on the application that exposes metrics, `/metrics/`. The application has been instrumented with `django_prometheus` to provide these metrics.

Prometheus is launched using Docker and configured via *prometheus.yml* file, exposing Prometheus on port 9090.

5.7.2 Visualization using Grafana

Grafana is an open-source analytics and interactive visualization web application. It is highly extensible and provides tools to build dashboards for visualizing data from various sources, including Prometheus. Grafana supports a wide range of chart types and is used to create real-time, dynamic dashboards that help monitor and analyze system performance.

Grafana is crucial for interpreting the raw data collected by Prometheus. It connects to Prometheus, queries its time-series database, and presents the data in an accessible, visual format.

Grafana was deployed using Docker and configured to run on port 3001, depending on Prometheus being available to function correctly.

A dashboard was created featuring several metrics, as exemplified in Figure 5.13. This allows for a comprehensive understanding of the application's behavior, facilitating detailed performance analysis and issue detection.

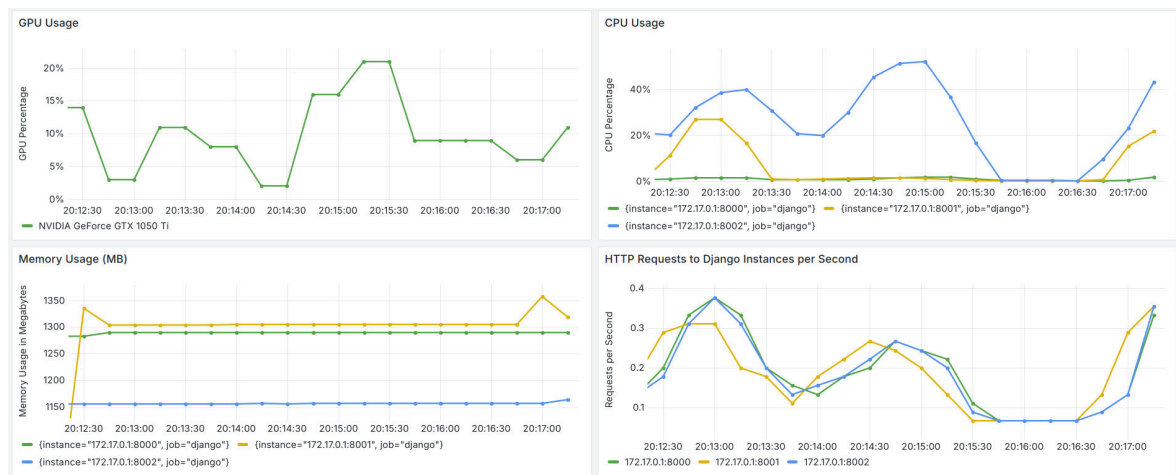


Figure 5.13: Grafana Dashboard

5.8 IMPLEMENTATION CHALLENGES

Implementing real-time weapon analysis for surveillance systems presents a series of significant challenges, primarily centered on balancing the trade-offs between speed, resolution, and system resources.

One of the difficulties arises from the key requirement to analyze and stream video content with minimal delay. Real-time processing is essential for effective surveillance, allowing for immediate response to potential security incidents as they occur. However, achieving this level of promptness often necessitates compromises in other aspects of the system.

A critical compromise involves the resolution of the video footage. To facilitate faster processing and streaming, it is often necessary to reduce the resolution of the video. This reduction speeds up the analysis by reducing the data load on the system, thus minimizing

the delay in video streaming. However, lower resolution images can significantly degrade the quality of the surveillance footage.

Furthermore, the scalability of real-time video analysis systems is another major challenge, particularly when multiple cameras are integrated into the surveillance network. As the number of cameras increases, so too does the overload on the system's computational resources, including CPU and GPU capacities. This escalation can lead to slower processing times and increased delays in video analysis and streaming, thereby compromising the real-time capability of the system.

5.9 SCALABILITY AND PERFORMANCE OPTIMIZATIONS

To ensure the system can handle increasing workloads and deliver optimal performance, several scalability and performance optimizations were implemented across all components.

In the frontend, Web Workers were utilized to handle frame processing in the background, offloading intensive tasks from the main thread and keeping the user interface responsive and fluid. Additionally, the number of Document Object Model (DOM) updates in canvas components was reduced by batching multiple updates and limiting unnecessary interactions with the DOM. This approach achieved smoother rendering and improved visual performance. By managing state effectively, unnecessary calls to external services were minimized. For example, instead of making multiple calls to RabbitMQ, a single persistent connection was established.

On the backend, a dedicated worker pool was employed to handle frame processing in parallel. This setup took advantage of multi-core processors, distributing the workload among multiple workers and significantly increasing the system's throughput and scalability. Unused memory was periodically cleared to prevent memory leaks and maintain efficient memory usage. The frequency of frames analyzed by the object detection algorithm was reduced, processing every 10th frame instead of every frame to balance performance with accuracy and reduce computational demands without significantly compromising detection capabilities. Batch processing for frames was also implemented, allowing multiple frames to be processed simultaneously rather than individually, thus maximizing the GPU's parallel processing power.

For RabbitMQ various settings were optimized to achieve better performance. The prefetch count was increased to allow RabbitMQ to send more messages to consumers without waiting for acknowledgments, reducing latency and improving throughput.

Tests and Results

This chapter presents the performance metrics of the object detection model, supported by video results showcasing its effectiveness across various scenarios. Moreover, the user experience is evaluated with end users, specifically police officers, providing valuable feedback.

6.1 YOLO MODEL RESULTS EVALUATION

The performance evaluation of the object detection model was conducted using a computer equipped with an NVIDIA Quadro RTX 8000 GPU. This GPU, renowned for its high-performance capabilities in professional visualization and computational tasks, features 48GB of GDDR6 memory. With a power usage capacity of 250W, the Quadro RTX 8000 is designed to handle demanding workloads, making it ideal for deep learning applications.

Table 6.1 presents performance metrics for the YOLOv5 model. The metrics include accuracy, precision, and recall, evaluated across various configurations defined by three parameters: the number of epochs, batch size, and image size. There are some notes associated to the parameters:

- Parameter image size has an influence related to size, as larger image sizes generally enhance the model's performance. For instance, an image size of 640 consistently produces better results compared to 256 across all epochs and batch sizes;
- Relatively to batch size parameter, larger batch sizes (32) tend to show an improvement in accuracy and recall over smaller batch sizes (16), as they provide a more stable gradient estimate during training. For example, with 100 epochs and an image size of 640, the accuracy for batch size 32 is 0.973, compared to 0.970 for batch size 16. Precision differences between batch sizes are less pronounced but still notable.
- The time taken for each epoch varies with batch size. Each epoch takes approximately 6 minutes to run for a batch size of 16. This means for configurations with 50 epochs, the total training time was around 5 hours, while for 100 epochs, it was approximately 10 hours. For a batch size of 32, each epoch takes approximately 7 minutes to run.

Consequently, for 50 epochs, the total training time was about approximately 6 hours, and for 100 epochs, it was around 12 hours.

Table 6.1: Training Parameters and Performance Metrics

Epochs	Batch Size	Image Size	Accuracy	Precision	Recall
50	16	256	0.827	0.899	0.772
50	16	640	0.965	0.955	0.934
50	32	256	0.832	0.907	0.771
50	32	640	0.966	0.965	0.934
80	16	256	0.847	0.894	0.800
80	16	640	0.971	0.975	0.936
80	32	256	0.832	0.907	0.771
80	32	640	0.971	0.975	0.936
100	16	256	0.851	0.909	0.796
100	16	640	0.970	0.966	0.941
100	32	256	0.853	0.914	0.797
100	32	640	0.973	0.969	0.944

Recall is a crucial metric. It quantifies the number of true positive detections out of all actual positives. High recall is essential in weapon detection scenarios because failing to detect a weapon (false negative) can have severe consequences. A model with high recall ensures that most, if not all, weapons are detected, minimizing the risk of undetected threats.

The line chart 6.1 demonstrates that configurations with larger image sizes (640) consistently achieve higher recall values across all epochs and batch sizes. The maximum recall value is 0.944, achieved with a batch size of 32 and an image size of 640.

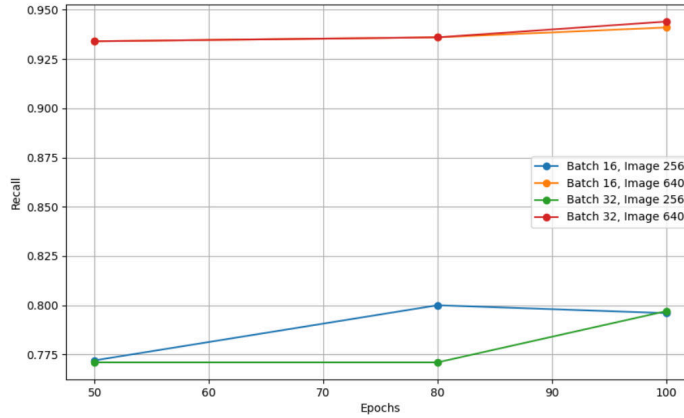


Figure 6.1: Recall Variation with Epochs

Precision, on the other hand, measures the number of true positive detections out of all positive predictions made by the model. While high precision reduces the number of false positives (incorrectly identified weapons), the balance between recall and precision is vital. In real-time weapon detection, a slightly lower precision might be acceptable if it results

in a significantly higher recall. This trade-off ensures that the system prioritizes safety by detecting as many weapons as possible, even at the cost of occasional false alarms.

Considering the analysis of the metrics and training times, the optimal configuration is represented in the Table 6.2.

Table 6.2: Optimal Configuration Parameters and Results

Epochs	Batch Size	Image Size	Accuracy	Precision	Recall
100	32	640	0.973	0.969	0.944

Despite longer training time, the improved performance justifies the use of a larger batch size and image size. The high recall ensures that most relevant instances are detected. Simultaneously, the high precision minimizes false positives, maintaining the system’s reliability and efficiency.

Figure 6.2 shows the Precision-Recall Curve for the optimal configuration of the trained model. This curve illustrates the trade-off between precision and recall for different classes of objects, being useful for understanding the performance of the model in distinguishing between different classes at various threshold levels. The area under the precision-recall curve (AP) for the weapon class is 0.953. The AP for the knife class is significantly higher at 0.991. The mAP across all classes at an IoU threshold of 0.5 is 0.972. The curve indicates that the model performs well in detecting knives, followed by general weapon detection, with overall high precision and recall values across all classes.

The Figure 6.3 shows how the F1 score, which is the harmonic mean of precision and recall, varies with the confidence threshold used for classifying detections. The maximum F1 score for the weapon class is around 0.96. The knife class also achieves a high F1 score, slightly above the weapon class. The F1 score for all classes combined reaches 0.96 at a confidence threshold of approximately 0.392. This curve demonstrates that the model maintains a high F1 score over a range of confidence thresholds, indicating that the model is reliable and robust in detecting weapons and knives with balanced precision and recall.

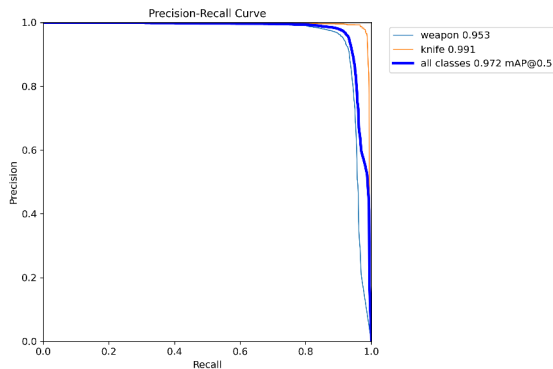


Figure 6.2: Precision-Recall Curve

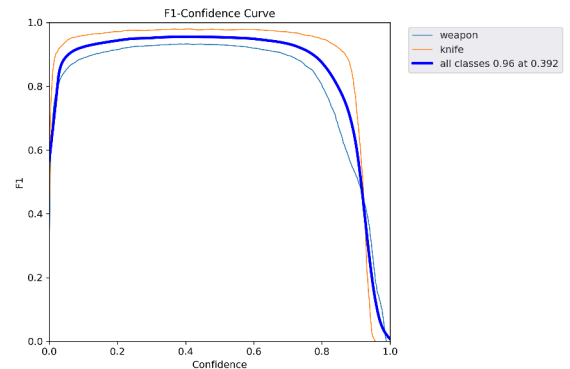


Figure 6.3: F1-Confidence Curve

Figure 6.4 is the confusion matrix for a the YOLOv5 model. It has three categories along both axes: weapon, knife, and background. The rows represent the predicted labels, while the

columns represent the true labels. The intensity of the color indicates the number of instances, with darker colors representing higher values.

For the weapon class, the model correctly predicted 93% of the instances, as indicated by the dark blue square in the top-left corner. The rest of the weapon instances were misclassified as background, as shown by the light blue square in the bottom-left corner. Additionally, no instances of weapons were misclassified as knives.

For the knife class, the model correctly predicted 98% of the instances, as shown by the dark blue square in the center of the matrix. The remaining 2% were misclassified as background, which is shown by the light blue square in the bottom-center section.

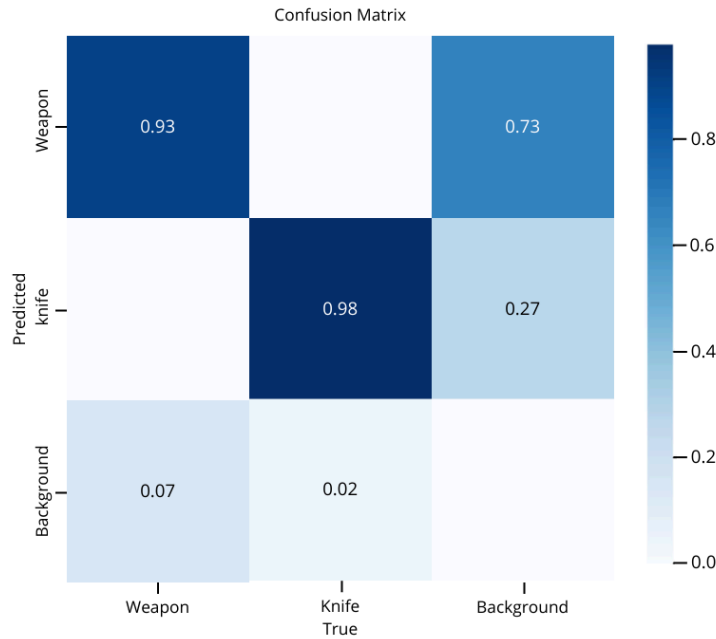


Figure 6.4: Confusion Matrix

6.2 REAL-WORLD SCENARIO TESTING

6.2.1 Non-Weaponized Scenario Analysis

The first test scenario is a 9-second video clip featuring a single human subject walking across a room while holding a cellphone, as shown in frame 6.5. The environment is devoid of any weapons. This scenario was chosen to evaluate the model's precision in identifying the absence of weapons and ensuring that non-threatening objects, such as cellphones, are not incorrectly detected as weapons.

The object detection model performed optimally in this non-weaponized scenario. Throughout the entire duration of the video, the model consistently refrained from flagging the cellphone or any other elements within the room as weapons. This result is indicative of the model's accuracy and reliability in distinguishing between actual threats and benign objects.

Another video was utilized to further assess the model's performance, exemplified in Figure 6.6. This test scenario involved a 10-second video featuring a human subject holding a water

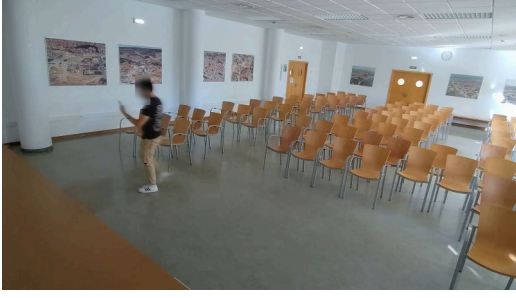


Figure 6.5: Video 1 Extracted Frame [44]

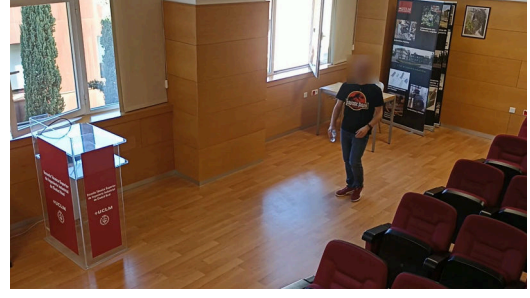


Figure 6.6: Video 2 Extracted Frame [44]

bottle inside a classroom. Similar to the previous scenario, the environment was free from any weapons, aiming to verify the model's capability to accurately identify non-threatening objects and avoid false positives.

6.2.2 Weaponized Environment Detection

The YOLOv5 model was applied to a 10-second video containing weapons. The video consists of 300 frames, approximately 30 frames per second, serving as the test sample.

Figure 6.7 shows a frame annotated by the object detection algorithm, while Figure 6.8 shows the same frame without being annotated.



Figure 6.7: Annotated Frame from Weaponized Environment [44]



Figure 6.8: Frame From Weaponized Scenario [44]

Out of the 300 frames, 196 frames contained objects of interest, in this case weapons. The YOLOv5 model successfully detected weapons in 131 of these frames. However, there were 69 frames containing weapons that the model failed to annotate. Given that 1 second of video has approximately 30 frames and the changes within this short duration are minimal, the model demonstrates a reasonable efficiency in identifying objects.

Some frames were not detected because the objects, being dark, were situated against a similarly dark background. Additionally, the human face in the video is completely blurred, and in some instances, the weapon is obscured behind the blurred face, further complicating detection.

6.3 USER EXPERIENCE EVALUATION

To ensure the effectiveness and usability of the application, was organized a presentation and demonstration session with the domain expert.

During this session, all the features and functionalities of the application were presented, showcasing its ability to detect weapons and provide immediate alerts. The interface, designed with simplicity, was demonstrated to highlight its intuitive navigation and clean layout.

The domain expert provided valuable feedback on the application. Overall, the feedback was positive, observing that the application was intuitive, making it easy for users to navigate and operate without extensive training. In addition to the positive remarks, offered constructive suggestions for further improvements. One proposed improvement was for the system to be able to upload more than one video at a time.

6.4 COMPARATIVE ANALYSIS

The performance of the pre-trained YOLOv5 model demonstrates superior accuracy compared to the majority of models discussed in the related work section. Specifically, the YOLOv5 model consistently achieves higher precision and recall rates, indicating its robustness in object detection tasks.

Our solution, in Figure 6.9, highlighted in red with an accuracy of 0.97, clearly outperforms other models, underscoring its effectiveness and reliability in real-time surveillance applications. The best performance among the other works is an accuracy of approximately 0.93.

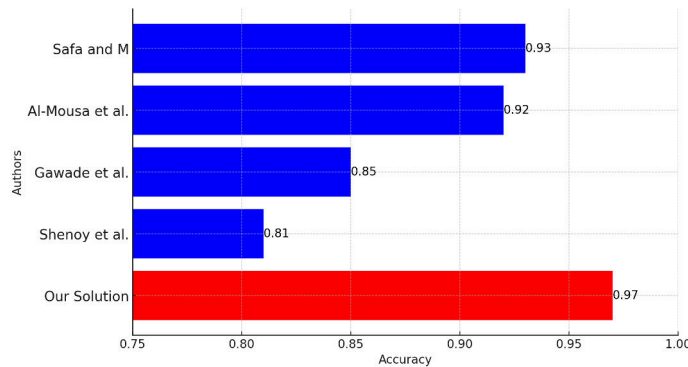


Figure 6.9: Comparative Analysis with Related Works

This system is a pioneering real-time weapon detection system designed for use by any police department to enhance the safety. It integrates a pre-trained object detection model, which has been tested on an extensive dataset, ensuring accurate identification of weapons in various scenarios. The model is integrated into a user-friendly web interface, enabling officers to access real-time insights and alerts directly from their devices.

As this detection system is a pioneering initiative, there is room for improvement. Continuous updates are necessary to enhance its accuracy, reduce false positives, and adapt to evolving threats. Feedback from the officers who will be using the system will be crucial in identifying areas for enhancement and ensuring the technology meets the practical needs of law enforcement. Additionally, expanding the dataset with more diverse and real-world examples can further train the model to handle a broader range of scenarios, ultimately improving its robustness and reliability.

Conclusions and Future Work

This final chapter summarizes the work, emphasizing the key findings. It presents the main conclusions drawn and outlines potential directions for future research.

7.1 CONCLUSIONS

The aim of this work was to develop a real-time web application for the detection of weapons and the triggering of alarms. The trained model achieved an accuracy of 0.973, coupled with high precision and recall. This ensured reliable and consistent performance in real-world scenarios.

This thesis was initially supported by a review of the literature related to the domain, and a review of the concepts and technology related to the domain. A crucial task was to define the dataset. We combined three datasets with small adjustments, such as the DaSCI Dataset [35]. Next, the model was trained using the previous custom dataset, defining the hyperparameters in order to obtain the best result. The model was used in an application created to orchestrate the user interface, the model, the database and the videos for analysis.

The integration of this pretrained model into a web application has significant practical implications. The application consumes data in real time, enabling it to detect and trigger alerts. Each detected frame is saved for further analysis, which is crucial for monitoring and responding to events as they unfold.

In real-world settings, this application can be used to monitor security cameras. The ability to process data in real time and trigger alerts can significantly influence practices in these fields by improving response times and enabling proactive measures. For instance, the system can alert personnel to potential threats instantly, allowing for rapid intervention.

Despite its successes, this study has limitations. While the model's performance is high, it encounters difficulties in specific scenarios. One significant limitation is the detection of weapons when they are partially concealed, such as in an attacker's pocket. The model's ability to identify threats is also challenged when the weapon is on a dark background, where contrast and visibility are reduced. Although the dataset includes images of these challenging

scenarios, the model can still fail under these conditions. Additionally, the real-time processing capability, although robust, may face challenges with extremely high data throughput or in environments with limited computational resources.

7.2 WORK CONTRIBUTES

This work makes significant contributions across several areas:

- Dataset development: involved the aggregation, data augmentation, and annotation of the dataset, in a format compatible with YOLO models. It was created from existing datasets.
- YOLO model creation: a YOLO model was developed, showing better results compared to related works;
- Functional prototype: scalable prototype named SafeGuard, capable of real-time weapon detection, was successfully designed. This prototype integrates the previous YOLO model, providing immediate and reliable detection results and allowing for the monitoring of multiple cameras simultaneously.
- Scientific publication: a paper was accepted for publication in the CISTI'2024, the 19th Iberian Conference on Information Systems and Technologies [57];
- Future publication: preparations are underway for an additional publication that will include updated results and a comprehensive overview of the fully developed prototype.

7.3 FUTURE WORK

The current implementation of this surveillance system provides a robust platform for monitoring through multiple camera feeds. However, to enhance its utility and performance, several improvements are proposed for future iterations.

- Implementation of Auto-Scaling Mechanisms: allow the system to dynamically adjust its resources based on workload demands. This will involve integrating monitoring tools and automated scaling policies. Additionally, exploring container orchestration platforms like Kubernetes will be essential for managing resource allocation and scaling efficiently.
- Upgrading System Hardware: the addition of real-time object detection across multiple feeds needs a corresponding upgrade in system hardware. As the number of cameras and the computational load increase, there is a substantial impact on system performance. To mitigate this, an enhancement in both CPU and GPU capabilities will be required. This upgrade will ensure that the system can handle the increased processing demands without degradation in performance;
- Exploration of Advanced Object Detection Models: the implementation of newer versions of the YOLO algorithm should be explored. These versions can offer improvements in detection accuracy;
- Dataset Adaptation: change the dataset according to police security camera footages and settings. This will involve customizing the dataset to reflect the specific environments and conditions encountered.

References

- [1] R. Shenoy, D. Yadav, H. Lakhotiya, and J. Sisodia, *An intelligent framework for crime prediction using behavioural tracking and motion analysis*, Internet Engineering Task Force, Mar. 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9758281>.
- [2] S. Gawade, D. R. Vidhya, and D. R. Radhika, *Automatic weapon detection for surveillance applications*, Internet Engineering Task Force, Jun. 2022. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4143822.
- [3] A. Al-Mousa, O. Z. Alzaibaq, and Y. K. A. Hashyeh, *Deep learning-based real-time weapon detection system*, International Journal of Computing and Digital Systems, Aug. 2023. [Online]. Available: <https://journal.uob.edu.bh/handle/123456789/4896>.
- [4] N. Hnoohom, P. Chotivatuny, N. Maitrichit, V. Sornlertlamvanich, S. Mekruksavanich, and A. Jitpattanakul, *Weapon detection using faster r-cnn inceptionv2 for a cctv surveillance system*, IEEE, Sep. 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9684649>.
- [5] M. T. Bhatti, M. Khan, M. Aslam, and M. J. Fiaz, *Weapon detection in real-time cctv videos using deep learning*, Internet Engineering Task Force, Mar. 2021. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9353483>.
- [6] M. Moran, A. Conci, and Á. Sánchez, *Automatic detection of knives in complex scenes*, Research Gate, Sep. 2022. [Online]. Available: https://www.researchgate.net/publication/363407966_Automatic_Detection_of_Knives_in_Complex_Scenes.
- [7] FBI, *Active shooter incidents in the united states in 2022*, FBI, Apr. 2023. [Online]. Available: <https://www.fbi.gov/file-repository/active-shooter-incidents-in-the-us-2022-042623.pdf/view>.
- [8] ATF, *Rules and regulations*, ATF, Oct. 2022. [Online]. Available: <https://www.atf.gov/rules-and-regulations/gun-control-act>.
- [9] E. Commission, *Eu legislation on civilian firearms*, European Commission. [Online]. Available: https://home-affairs.ec.europa.eu/policies/internal-security/organised-crime-and-human-trafficking/trafficking-firearms/eu-legislation-civilian-firearms_en.
- [10] *World population review*. [Online]. Available: <https://worldpopulationreview.com/country-rankings/gun-ownership-by-country>.
- [11] S.-G. do Sistema de Segurança Interna, *Relatório anual de segurança interna*, Sep. 2022. [Online]. Available: https://www.ssi.gov.pt/sites/default/files/2023-07/20230331_pm_RASI_2022.pdf.
- [12] E. L. Piza, B. C. Welsh, D. P. Farrington, and A. L. Thomas, *Cctv surveillance for crime prevention*, Wiley, 2019. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/1745-9133.12419#:~:text=Abstract%20Research%20Summary%20We%20report,were%20observed%20in%20car%20parks>.
- [13] S. C. Daily, *The effectiveness of cctv in reducing crime: Florida's experience explained*, Space Coast Daily, Nov. 2023. [Online]. Available: <https://spacecoastdaily.com/2023/11/the-effectiveness-of-cctv-in-reducing-crime-floridas-experience-explained/#:~:text=But%20where%20does%20Florida%20fit,according%20to%20certain%20studies>.
- [14] P. Matczak, A. Wójtowicz, A. Dąbrowski, and K. Mączka, *Cost-effectiveness of cctv surveillance systems: Evidence from a polish city*, European Journal on Criminal Policy and Research, Sep. 2022. [Online]. Available: <https://link.springer.com/article/10.1007/s10610-022-09527-5>.

- [15] B. C. Welsh, E. L. Piza, A. L. Thomas, and D. P. Farrington, *Private security and closed-circuit television (cctv) surveillance: A systematic review of function and performance*, Journal of Contemporary Criminal Justice, Feb. 2020. [Online]. Available: <https://journals.sagepub.com/doi/epdf/10.1177/1043986219890192?src=getftr>.
- [16] M. Gerell, *Cctv in deprived neighbourhoods - a short-time follow-up of effects on crime and crime clearance*, Department of Criminology, Malmö University, Jul. 2021. [Online]. Available: <https://www.idunn.no/doi/full/10.1080/2578983X.2020.1816023>.
- [17] H. Keval and M. A. Sasse, “*not the usual suspects*” : *A study of factors reducing the effectiveness of cctv*, Wiley, 2010. [Online]. Available: <https://link.springer.com/article/10.1057/palgrave.sj.8350092>.
- [18] R. V. G. Gaylon, R. A. Galapia, R. C. Mabborang, and A. G. Bansi, *A proposed optimization algorithm for solving cctv camera placement*, Nov. 2022. [Online]. Available: <https://www.ej-compute.org/index.php/compute/article/view/75>.
- [19] Z.-Q. Zhao, P. Zheng, S.-t. Xu, and X. Wu, *Object detection with deep learning: A review*, Internet Engineering Task Force, Apr. 2019. [Online]. Available: <https://arxiv.org/abs/1807.05511>.
- [20] R. Kaur and S. Singh, *A comprehensive review of object detection with deep learning*, Internet Engineering Task Force, Jan. 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1051200422004298?ref=cra_js_challenge&fr=RR-1.
- [21] G. L. Team, *Yolo object detection using opencv*, visited on 21/10/2023, Mar. 2022. [Online]. Available: <https://www.mygreatlearning.com/blog/yolo-object-detection-using-opencv/>.
- [22] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, *Object detection in 20 years: A survey*, IEEE, Jan. 2023. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10028728>.
- [23] P. Soviany and R. T. Ionescu, *Optimizing the trade-off between single-stage and two-stage deep object detectors using image difficulty prediction*, arxiv, Aug. 2018. [Online]. Available: <https://arxiv.org/pdf/1803.08707.pdf>.
- [24] M. Carranza-García, J. Torres-Mateo, P. Lara-Benítez, and J. García-Gutiérrez, *On the performance of one-stage and two-stage object detectors in autonomous vehicles using camera data*, MDPI, Dec. 2020. [Online]. Available: <https://www.mdpi.com/2072-4292/13/1/89>.
- [25] R. Padilla, W. L. Passos, T. L. B. Dias, S. L. Netto, and E. A. B. da Silva, *A comparative analysis of object detection metrics with a companion open-source toolkit*, MDPI, Jan. 2021. [Online]. Available: <https://www.mdpi.com/2079-9292/10/3/279>.
- [26] R. Padilla, S. L. Netto, and E. A. B. da Silva, *A survey on performance metrics for object-detection algorithms*, IEEE, Jul. 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/9145130>.
- [27] S. Srivastava, A. V. Divekar, C. Anilkumar, I. Naik, V. Kulkarni, and V. Pattabiraman, *Comparative analysis of deep learning image detection algorithms*, Internet Engineering Task Force, May 2021. [Online]. Available: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-021-00434-w>.
- [28] C. Dataset. [Online]. Available: <https://cocodataset.org/#home>.
- [29] *Pascal voc dataset*, Accessed: Jan 2024. [Online]. Available: <http://host.robots.ox.ac.uk/pascal/VOC/>.
- [30] L. Alzubaidi, J. Zhang, A. J. Humaidi, et al., *Review of deep learning: Concepts, cnn architectures, challenges, applications, future directions*, Internet Engineering Task Force, Jan. 2021. [Online]. Available: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-021-00444-8>.
- [31] A. Rehman and L. G. Fahad, *Real-time detection of knives and firearms using deep learning*, Internet Engineering Task Force, Dec. 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9972915>.
- [32] *Imfdb dataset*, Accessed: Jan 2024. [Online]. Available: https://imfdb.org/wiki/Main_Page.

- [33] R. Thangaraj, P. T. P. P., U. R. M., and B. vadivelu, *Real-time handgun detection in surveillance videos based on deep learning approach*, Internet Engineering Task Force, May 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9793288>.
- [34] *Google collab*, Accessed: Jan 2024. [Online]. Available: <https://colab.google/>.
- [35] *Dasci dataset*, Accessed: Jan 2024. [Online]. Available: <https://dasci.es/transferencia/open-data/24705/>.
- [36] *Caviar dataset*, Accessed: Jan 2024. [Online]. Available: <https://homepages.inf.ed.ac.uk/rbf/CAVIARDATA1/>.
- [37] F. Safa and S. M., *Subduing crime and threat in real-time by detecting weapons using yolov8*, IEEE, Aug. 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/10245146>.
- [38] *StarUML*, Accessed: Jun 2024. [Online]. Available: <https://staruml.io/>.
- [39] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, *You only look once: Unified, real-time object detection*, arxiv, Jun. 2015. [Online]. Available: <https://arxiv.org/abs/1506.02640>.
- [40] J. Terven and D. Cordova-Esparza, *A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas*, arxiv, Feb. 2024. [Online]. Available: <https://arxiv.org/abs/2304.00501>.
- [41] G. Lavanya and S. D. Pande, *Enhancing real-time object detection with yolo algorithm*, Research Gate, Dec. 2023. [Online]. Available: https://www.researchgate.net/publication/376252973_Enhancing_Real-time_Object_Detection_with_YOLO_Algorithm.
- [42] R. Xu, H. Lin, K. Lu, and L. Cao, *A forest fire detection system based on ensemble learning*, Research Gate, Feb. 2021. [Online]. Available: https://www.researchgate.net/publication/349299852_A_Forest_Fire_Detection_System_Based_on_Ensemble_Learning.
- [43] *Mock attack dataset*. [Online]. Available: <https://github.com/Deepknowledge-US/US-Real-time-gun-detection-in-CCTV-An-open-problem-dataset>.
- [44] J. Ruiz-Santaquiteria, J. D. Muñoz, F. J. Maigler, Ó. Déniz, and G. Bueno, *Action recognition and object detection dataset for firearm-related actions*, Nov. 2023. [Online]. Available: <https://data.mendeley.com/datasets/bbzpxhd22j/2>.
- [45] *Rabbitmq*, Accessed: Jun 2024. [Online]. Available: <https://www.rabbitmq.com/>.
- [46] *Django official website*, Accessed: Jun 2024. [Online]. Available: <https://www.djangoproject.com/>.
- [47] *Mysql*, Accessed: Jun 2024. [Online]. Available: <https://www.mysql.com/>.
- [48] *Nginx*, Accessed: Jun 2024. [Online]. Available: <https://nginx.org/en/>.
- [49] *React official website*, Accessed: Jun 2024. [Online]. Available: <https://react.dev/>.
- [50] *Prometheus*, Accessed: Jun 2024. [Online]. Available: <https://prometheus.io/>.
- [51] *Grafana labs*, Accessed: Jun 2024. [Online]. Available: <https://grafana.com/>.
- [52] *Microservices in python: Django, rabbitmq and pika*, Accessed: Mar 2024. [Online]. Available: <https://medium.com/@mansha99/microservices-in-python-django-rabbitmq-and-pika-feladb0c6a1a>.
- [53] *Opencv*, Accessed: Jun 2024. [Online]. Available: <https://pypi.org/project/opencv-python/>.
- [54] *Jest*, Accessed: Jun 2024. [Online]. Available: <https://jestjs.io/>.
- [55] *Selenium*, Accessed: Jun 2024. [Online]. Available: <https://www.selenium.dev/>.
- [56] *Firefox web driver*, Accessed: Jun 2024. [Online]. Available: <https://github.com/mozilla/geckodriver/releases>.
- [57] *Cisti'2024 - 19th iberian conference on information systems and technologies*, 25th and 28th of June 2024. [Online]. Available: <https://cisti.eu/index.php/en/>.

